Ismail's Primitives

A Unified Functional Theory of Necessity, Independence, and Sequential Dependence in Adaptive Decision Systems

Muhammed Ismail
Independent Researcher
Email: literacity@outlook.com
ORCID: 0009-0000-3713-7105

July 3, 2026

This paper asks one question: what must any decision-maker — a person, an institution, or a machine — get right before sustained success under real uncertainty is even possible? Across investing, operations, public policy, and machine learning, the same six structural failures keep recurring: ambiguity nobody resolves, mistakes nobody can undo, comfortable plateaus nobody leaves, decisions nobody commits to, limits nobody respects, and beliefs nobody updates. The usual response is a patchwork — a regret bound here, a compliance checklist there, a risk framework somewhere else — each useful, none transferable, none provably complete. This paper instead asks what is unconditionally true of any sequential decision process, of any kind, anywhere. The answer is six capabilities, and only six (named X_1 through X_6 in the body): tracking the true objective, avoiding the mistake you cannot undo, exploring past the comfortable plateau, committing once the evidence is in, respecting the limits that cannot be traded away, and updating when the world changes. Lacking any one of them is not a stylistic weakness; it is a mathematical guarantee of failure that compounds the longer the decision-maker operates — and no other one of the six can substitute for it. None of this is asserted on faith. Every claim, including that no primitive can stand in for another, is verified line by line in the Lean 4 proof assistant, a computer that accepts nothing it cannot check itself; the same six results hold, unmodified, whether the decision-maker is a trading desk, a regulator, a hospital, or a reinforcement-learning agent. Each new domain tested is not merely another example: it returns evidence to the same six claims — evidence that has so far only confirmed them — and the theory invites the next test just as openly as the last.

Author's note. The framework, every definition, every theorem statement and proof, and the complete Lean 4/Mathlib formalization underlying this paper are the sole original work of the author — conceived, proved, and machine-verified without co-authors, institutional affiliation, or external funding. The complete formalization is available at `https://github.com/M-Ismail-ZA/IsmailsPrimitives`.

A note on structure. The six movements of the abstract above — stating the question, naming the failures, rejecting the patchwork, committing to six capabilities, holding to a non-negotiable verification standard, and closing on a claim that strengthens with each new test — follow the six primitives in order. This is not ornamental: writing a clear, falsifiable claim about decision-making is itself a sequential decision under uncertainty, so doing it well takes the same six things.

Keywords: sequential decision-making; necessity and sufficiency; impossibility theorems; partially observable Markov decision processes; regret bounds; formal verification; Lean 4; cross-domain transfer

Plain-Language Overview

Imagine equipping a rover for a mission without knowing which planet it will land on, or whether it faces extreme heat, corrosive radiation, or treacherous terrain. The only sound engineering choice is to prepare for every danger you can characterize in advance, even though any single mission will likely meet only a few. This paper does the same for decision-making systems — in investing, operations, public policy, or machine learning — by identifying six structural failures that recur across all of them, and asking exactly three questions about each.

Is it necessary? For each of the six structural failures, I build the smallest possible test environment that contains it — one hidden choice, one trap, one tempting-but-mediocre plateau — and prove that any decision process without the matching capability fails that test, permanently and increasingly, not just occasionally. The environments are deliberately toy-like: if a process cannot pass the simplest version of a failure, it has no chance against the messy version that shows up in the real world.

Is it replaceable? No. Being good at handling one of the six structural failures does not make a process any better at handling a different one: a model built to flag fraud is not, by virtue of that skill, also a model for regulatory-capital allocation. Each capability has to be separately earned; none can stand in for another.

Do they work together? This is the most delicate claim in the paper. The six capabilities assemble into a single forward chain, where having one creates exactly the conditions needed to benefit from the next, looping back at the end so that evidence compounds over repeated cycles rather than resetting. Two of the six links, and parts of several others, still need domain-specific work before the chain's promise is fully cashed out for any given real system: I built the verified scaffolding for that chain and show precisely where the remaining work lies.

Everything past this point is the rigorous version of these same three questions, stated in the language of probability and information theory so every claim can be, and has been, checked by a computer proof assistant rather than taken on faith. Section 1 restates the motivation formally and gives a table translating each capability into investing, operations, and policy language for readers who want to keep reading that way throughout.

Contents

1 Introduction

1.1 Why a Unified Functional Theory

Most accounts of what makes a sequential decision process trustworthy are either too narrow or too vague to be portable. A bandit-algorithm regret bound is precise but speaks only to bandits. A risk-management checklist is portable but unfalsifiable — nothing in it is stated sharply enough to be proved false. This paper takes a third path, and names it accordingly: a *Unified Functional Theory* — *unified* because the same small number of structural failures recur across investing, operations, public policy, and reinforcement learning alike, stated abstractly enough that one proof serves all of them at once; *functional* because what is characterized is what a decision process must *do* under uncertainty, never what field it belongs to, what institution runs it, or what it calls itself. For each structural failure, this paper proves an unconditional floor (Part I), a demonstration that none of the others can substitute for it (Part II), and a fully specified — if partly conditional — account of how the six combine into a single forward pipeline (Part III). The six properties that result are referred to throughout as X_1 through X_6, and the paper's central empirical claim is narrow and falsifiable by construction: each is necessary in a minimal instance of its matching structural failure, and no other one of the six can stand in for it there.

1.2 Roadblocks, Not Recipes

Each Part of this paper makes a different kind of claim; treating them as interchangeable is exactly the failure mode this paper exists to diagnose.

Part I places six roadblocks: a roadblock is a minimal condition whose absence guarantees failure regardless of what else a decision process does well — it marks the cliff edge, not a blueprint. Part II proves the six are non-substitutable for every pair, not merely assumed so.

Part III is the only part describing how to move from *not failing* to *actually succeeding*: it requires all six primitives combined in one specific order, not any subset and not an arbitrary sequence. Clearing every roadblock does not by itself describe a path forward; the sequential-dependence chain of Part III is the closest thing this paper offers to one, and states plainly, link by link (Section 8.1), which parts of that bridge are fully built and which are left for a concrete implementation to supply.

This structure also fixes what it means to port this theory into another field. Porting one primitive alone — noticing that decision-makers under uncertainty need good objective-tracking, say — is not the claim this paper makes; it is standard decision theory, studied under its own name for decades. Porting the full union, with the proof that none of the six substitutes for another and the proof of the one sequence that combines them into sufficiency, is a considerably stronger claim, taken up directly in Section 4.3.

1.3 Discovered, Not Designed

One claim in this paper is easy to state and impossible to dispute once its premise is granted: if a decision process genuinely faces one of the six structural failures of Section 3 — in the precise, checkable sense defined there, not a loose family resemblance — then the matching primitive is necessary for it, full stop, regardless of field or vocabulary. This is the ordinary behavior of every lower bound and impossibility theorem there has ever been: Arrow's theorem does not care what a voting system calls itself, and the Halting Problem does not care what language a program was written in. A real decision process either has the structure or it doesn't, and no redescription changes which.

That premise — that a given real situation actually has the structure — is a different kind of claim, kept separate here from the necessity result itself. Section 4 argues, with the same evidentiary standard economists have used since Akerlof to show that real markets violate frictionless-equilibrium idealizations, that realistic decision processes generically land in Class $\mathcal{C}$ — not by definitional fiat, but because the assumptions that would keep them out (complete information, no irreversible mistakes, no tempting local optima, total flexibility, stationarity) are exactly what economics, control theory, and decision theory have

spent decades documenting real markets, institutions, and choices under uncertainty as failing to satisfy. Together, the two claims justify reading every cross-domain example in this paper as substantive rather than decorative.

1.4 The Minimality Principle

Every environment in Part I is deliberately the smallest instance of its structural failure that still forces failure — one ambiguous state for X_1, one trap for X_2, one local optimum for X_3, and so on (Section 3.3 states this precisely). This is what licenses reading the results outside reinforcement learning: a necessity result proved on a stripped-down instance is a floor that persists into every messier, real environment containing that instance as a special case, whereas a result proved only on an elaborate instance might be an artifact of the elaboration. The paper's transferability claim rests on this one structural choice, not on an appeal to analogy.

1.5 Reading This Paper Outside Reinforcement Learning

The formal apparatus — POMDPs, regret, mutual information — is necessary for the proofs to be machine-checked, but the six properties it characterizes are not specific to reinforcement learning. Table 1 gives one fixed cross-domain reading for each; every subsequent section develops its own primitive's reading in more depth via boxed asides, but the table is what a reader returning to a single section out of order should consult first.

Primitive	**Plain meaning**	**Outside reinforcement learning**
X_1 Objective Tracking	Tell two live hypotheses apart when they prescribe different actions	A fund that can't distinguish which of two strategies is actually working; a clinical trial that can't tell which arm is better
X_2 Cross-Context Safety Transfer	Stop touching the action that locks in permanent loss	A trading desk that won't stop revisiting a blow-up trade; an operation one safety violation away from losing its license
X_3 Global Attractor Exploration	Keep probing past a comfortable plateau toward a genuinely better region	A firm stuck at a mediocre-but-safe strategy with no scaling exploration budget; an agency that never pilots anything past a one-off budget
X_4 Policy Simplification	Commit decisively once the evidence supports one option	A committee still "deliberating" on a question the data settled quarters ago
X_5 Feasibility Projection	Respect a hard constraint rather than trade it off	A regulator-bound allocator that occasionally drifts into the forbidden range because it scored well elsewhere
X_6 Feedback Adaptation	Discard stale evidence once the world has changed	A strategy still sized for the regime that ended, or a policy still calibrated to a population that has moved on

Table 1: The six primitives outside reinforcement learning. Each row is developed further, with a worked example, in its own subsection of Part I.

1.6 Related Work

The necessity arguments of Part I are in the classical two-point / Le Cam testing tradition used throughout the bandit and sequential decision-making lower-bound literature, and the Fano-type argument behind X_4's

necessity proof is likewise standard in that tradition; Lattimore and Szepesvári (2020) is a comprehensive modern reference for both techniques and for the regret framework used throughout this paper. The formalization itself is carried out in Lean 4 (de Moura and Ullrich, 2021) on top of Mathlib (The mathlib Community, 2020), the community-maintained library of formalized mathematics that supplies the measure theory, probability, and analysis this paper's proofs are built from.

This paper's contribution relative to that literature is not a new algorithm or a new regret bound, but a different question: not "how fast can regret be driven to zero," but "which structural properties are unconditionally necessary, mutually irreplaceable, and combinable into a single verified pipeline, stated abstractly enough to apply outside reinforcement learning altogether."

1.7 Organization

Section 2 fixes notation. Section 3 defines the environment class $\mathcal{C}$ and its six structural failures. Section 4 argues that realistic decision processes generically belong to $\mathcal{C}$ and gives checkable recognition tests for each structural failure. Section 5 states the (small) amount of general-purpose machinery the rest of the paper actually uses. Part I (Section 6) proves each primitive necessary. Part II (Section 7) proves the six mutually independent. Part III (Section 8) gives the verified logical scaffold for how the six combine, stating plainly which hypotheses are discharged within the paper and which are left for instantiation. Section 9 collects what has and has not been shown; Section 10 closes. Appendix A maps every numbered result to its exact Lean source; Appendix B verifies class membership for every environment used.

2 Setup, Notation, and the Canonical Summary

2.1 Environments

Definition 2.1 (Environment)**.** *An* environment *on a measurable state space S, action space A, and observation space O consists of:*

- *a stochastic transition kernel* $\mathrm{trans} : S \times A \to \Delta(S)$*;*
- *a stochastic observation kernel* $\mathrm{obs} : S \times A \to \Delta(O)$*;*
- *a deterministic, measurable reward function* $r : S \times A \to \mathbb{R}$*;*
- *an initial-state probability measure μ_0 on S.*

We write $\mathrm{Env}(S, A, O)$ *for this type and $\mathcal{E}$ for a generic member of it.*

Lean 4 / Mathlib — `Phase0.lean, lines 32--45`

```
structure Env (S A O : Type*)
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O] where
  /-- Stochastic state transition: (s, a) →distribution over s' -/
  trans : Kernel (S ×A) S
  /-- Stochastic observation emission: (s, a) →distribution over o -/
  obs : Kernel (S ×A) O
  /-- Deterministic reward: r(s, a) ∈ℝ-/
  r : S ×A →ℝ
  /-- Measurability of r ---required for 'Kernel.deterministic' in Phase 2. -/
  hr_meas : Measurable r
  /-- Initial state distribution -/
  μ₀: Measure S
  /-- μ₀is a probability measure -/
  hμ₀ : IsProbabilityMeasure μ₀
```

Two structural conditions recur throughout the paper and are worth fixing notation for now.

Definition 2.2 (Markov and deterministic environments)**.** *$\mathcal{E}$ is* Markov *if* trans *and* obs *are genuine probability kernels (every fibre is a probability measure, not merely a sub-probability measure). $\mathcal{E}$ is* deterministic *if, in addition, μ_0 is a point mass and* trans, obs *are point masses at measurable functions of (s, a) — i.e. the environment's own dynamics involve no randomness, only the algorithm's choices do.*

2.2 Algorithms and the Canonical Summary

Definition 2.3 (Algorithm)**.** *An* algorithm *with summary type* Sig *consists of:*

- *an action kernel* act : $\mathrm{Sig} \to \Delta(A)$*;*
- *a summary-update kernel* update : $\mathrm{Sig} \times A \times O \times \mathbb{R} \to \Delta(\mathrm{Sig})$*;*
- *an initial summary $\sigma_0 \in \mathrm{Sig}$.*

Lean 4 / Mathlib — `Phase0.lean, lines 50--59`

```
structure Algorithm (A O Sig : Type*)
    [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig]
    [BorelSpace Sig] where
  /-- Action selection: depends only on summary σ. -/
  act : Kernel Sig A
  /-- Summary update: (σ, action, observation, reward) →new summary. -/
  update : Kernel (Sig ×A ×O ×ℝ) Sig
  /-- Initial summary. -/
  σ₀: Sig
```

The definition is deliberately restrictive in one respect: act takes *only* the current summary σ_t as input, never the raw history of past actions, observations, and rewards directly, and never the environment's hidden state. Whatever the algorithm has learned by time t must already be compressed into σ_t; there is no second channel by which the past can leak into the present decision. Every result in this paper is a statement about what shape that compression must take — it is never a statement about an algorithm with secret side-channel access to its own history.

Intuition — Operations

Think of σ_t as a manager's dashboard, refreshed every period. The manager's next decision can only depend on what the dashboard currently shows — not on the raw transaction log sitting in a filing cabinet she never reopens. If the dashboard is well designed, this costs nothing: a good summary keeps everything decision-relevant. If it is poorly designed, the manager will keep making the same correctable mistake, because the dashboard never surfaced the pattern in the first place. The six primitives are, at bottom, six different ways a dashboard can quietly fail to surface something it needed to.

2.3 Trajectories and the Trajectory Measure

Definition 2.4 (Trajectory)**.** *For $T \in \mathbb{N}$, a length-T* trajectory *is a function $\omega : \mathrm{Fin}(T) \to A \times O \times \mathbb{R}$, recording the action taken, observation received, and reward earned at each of the first T steps.*

Lean 4 / Mathlib — `Phase2.lean, line 202`

```
abbrev Trajectory (A O : Type*) (T : ℕ) := Fin T →A ×O ×ℝ
```

Running an algorithm in an environment for T steps induces a probability measure on length-T trajectories, built up one step at a time. Each step, from a current pair (σ_t, s_t), generates

$$a_t \sim \mathrm{act}(\sigma_t), \qquad o_t \sim \mathrm{obs}(s_t, a_t), \qquad r_t = r(s_t, a_t), \qquad s_{t+1} \sim \mathrm{trans}(s_t, a_t), \qquad \sigma_{t+1} \sim \mathrm{update}(\sigma_t, a_t, o_t, r_t),$$

with a_t, o_t, s_{t+1}, and σ_{t+1} conditionally independent given (σ_t, s_t, a_t) as appropriate, and the t-th coordinate of the trajectory set to (a_t, o_t, r_t). Notice that the observation, the reward, and the new state are all generated from the *same* pre-transition pair (s_t, a_t) — in particular the summary update sees o_t and r_t but never s_{t+1} directly, exactly as it must if the algorithm is to have no access to the environment's hidden state except through what it observes.

Definition 2.5 (Trajectory measure)**.** *For an environment $\mathcal{E}$, algorithm* alg*, and horizon T, the construction above defines a measure* $\mathrm{trajMeasure}(\mathcal{E}, \mathrm{alg}, T)$ *on* $\mathrm{Trajectory}(A, O, T)$*. When $\mathcal{E}$ is Markov and* alg*'s kernels are genuine probability kernels, this measure is a probability measure.*

Lean 4 / Mathlib — `Phase2.lean, lines 204--211`

```
noncomputable def trajMeasure
    (env : SixPrimitives.Env S A O)
    (alg : SixPrimitives.Algorithm A O Sig)
    (hr : Measurable env.r)
    (T : ℕ) : Measure (Trajectory A O T) :=
  let κT := trajMeasureAux env alg hr T
  let μ₀: Measure (Sig ×S) := (Measure.dirac alg.σ₀).prod env.μ₀
  (μ₀.bind κT).map Prod.fst
```

Remark 2.6. *The recursive per-step kernel (`oneStepKernel`, Phase2.lean lines 119–140) and the inductive construction that chains T copies of it together (`trajMeasureAux`, lines 142–200) are routine, if notationally heavy, measure-theoretic plumbing, omitted from the main text; see https://github.com/M-Ismail-ZA/IsmailsPrimitives.*

2.4 Value, Regret, and the Oracle Baseline

Definition 2.7 (Algorithm value and optimal value)**.** *The* value *of* alg *in $\mathcal{E}$ at horizon T is its expected cumulative reward,* $\mathrm{algValue}(\mathcal{E}, \mathrm{alg}, T) = \mathbb{E}_{\omega \sim \mathrm{trajMeasure}(\mathcal{E}, \mathrm{alg}, T)}\left[\sum_{t<T} \omega(t).r\right]$*. The* optimal value $\mathrm{optValue}(\mathcal{E}, T)$ *is the supremum of* $\mathrm{algValue}(\mathcal{E}, \mathrm{alg}, T)$ *over* every *summary type and every algorithm on it whose kernels are genuine probability kernels — not just over some fixed reference class.*

Intuition — Investing

optValue is not "the best fixed strategy" or "the best strategy from some pre-approved playbook." It is the best *any* conceivable strategy, of any complexity, could have achieved with full hindsight knowledge of the environment's dynamics — the same role a perfect-foresight benchmark plays when grading a fund manager. Defining the baseline this generously is what makes a regret bound meaningful: an algorithm cannot be let off the hook by comparing it only to a weak competitor.

Definition 2.8 (Regret)**.** $\mathrm{regret}(\mathcal{E}, \mathrm{alg}, T) := \mathrm{optValue}(\mathcal{E}, T) - \mathrm{algValue}(\mathcal{E}, \mathrm{alg}, T)$*.* alg *has* sublinear regret *in $\mathcal{E}$ if* $\mathrm{regret}(\mathcal{E}, \mathrm{alg}, T) \le \varepsilon T$ *eventually, for every $\varepsilon > 0$; it has* linear regret *if there is a fixed $c > 0$ with* $\mathrm{regret}(\mathcal{E}, \mathrm{alg}, T) \ge c(T - t_0)$ *eventually, for some t_0.*

Lean 4 / Mathlib — `Phase2.lean, lines 1600--1632`

```
noncomputable def algValue {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
```

```
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    (env : Env S A O) (alg : Algorithm A O Sig) (T : ℕ) : ℝ:=
  Phase2.algValue' env alg env.hr_meas T

noncomputable def optValue {S A O : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    (env : Env S A O) (T : ℕ) : ℝ:=
  sSup { v : ℝ| ∃(Sig : Type)
                 (_ : MeasurableSpace Sig) (_ : TopologicalSpace Sig) (_ : BorelSpace Sig)
                 (alg : Algorithm A O Sig)
                 (_ : IsMarkovKernel alg.act) (_ : IsMarkovKernel alg.update),
               v = Phase2.algValue' env alg env.hr_meas T }

noncomputable def regret {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    (env : Env S A O) (alg : Algorithm A O Sig) (T : ℕ) : ℝ:=
  optValue env T - algValue env alg T

def SublinearRegret {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    (env : Env S A O) (alg : Algorithm A O Sig) : Prop :=
  ∀ε> 0, ∀ᶠT : ℕin Filter.atTop, regret env alg T ≤ε* T

def LinearRegret {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    (env : Env S A O) (alg : Algorithm A O Sig) : Prop :=
  ∃c : ℝ, 0 < c ∧∃t₀ : ℕ,
    ∀ᶠT : ℕin Filter.atTop, c * (T - t₀) ≤regret env alg T
```

Remark 2.9. *Sublinear and linear regret are not exhaustive opposites stated this way — an algorithm could in principle have regret that oscillates between the two regimes forever. Necessity results (Part I) sidestep this by proving linear regret outright, which is always sufficient to rule out sublinearity.*

2.5 The Failure and Possession Hierarchy

Each primitive X_i is pinned down by a single real-valued sequence summarizing the algorithm's behaviour relevant to it (a mutual-information sequence for X_1, a danger-action probability sequence for X_2, and so on); which concrete sequence is plugged in depends on the environment and is fixed when each primitive is introduced in Part I. Independently of that choice, three nested behaviours are defined once, abstractly, over any such sequence.

Definition 2.10 (Failure conditions F_1–F_6)**.** *An algorithm, summarized by the relevant sequence,* lacks X_i *if:*

F_1 *its mutual-information sequence* mi *satisfies* $\limsup_t \mathrm{mi}(t) \le B$ *for some* $B < \frac{1}{2}$*;*

F_2 *its danger-probability sequence* dangerProb *satisfies* $\mathrm{dangerProb}(k) \ge \delta$ *for all* k*, for some fixed* $\delta > 0$*;*

F_3 *its cumulative bridge-count sequence* cumBridge *satisfies* $\mathrm{cumBridge}(T) \le C$ *for all* T*, for some fixed* $C > 0$*;*

F_4 *its conditional-entropy sequence* condEnt *satisfies* $\liminf_t \mathrm{condEnt}(t) \ge \varepsilon$ *for some fixed* $\varepsilon > 0$*;*

F_5 *its infeasible-play sequence* infeasProb *has Cesàro average satisfying* $\liminf_T T^{-1}\sum_{t<T}\text{infeasProb}(t) \geq \delta$ *for some fixed* $\delta > 0$*;*

F_6 *its stale-arm probability sequence* staleProb *satisfies* $\text{staleProb}(t) \geq \frac{1}{2} + \delta$ *throughout a nonempty post-changepoint window* $(\tau, (1+\varepsilon)\tau]$*, for some fixed* $\varepsilon \in (0, \frac{1}{2}]$ *and* $\delta > 0$*.*

Lean 4 / Mathlib — `Phase0.lean, lines 69--102`

```
def LacksX₁ (mi : ℕ→ℝ) : Prop :=
  ∃B : ℝ, B < 1/2 ∧limsup (fun t : ℕ=> mi t) atTop ≤B

def LacksX₂ (dangerProb : ℕ→ℝ) : Prop :=
  ∃δ: ℝ, 0 < δ∧∀k : ℕ, dangerProb k ≥δ

def LacksX₃ (cumBridge : ℕ→ℝ) : Prop :=
  ∃C : ℝ, 0 < C ∧∀T : ℕ, cumBridge T ≤C

def LacksX₄ (condEnt : ℕ→ℝ) : Prop :=
  ∃ε: ℝ, 0 < ε∧ε≤liminf (fun t : ℕ=> condEnt t) atTop

def LacksX₅ (infeasProb : ℕ→ℝ) : Prop :=
  ∃δ: ℝ, 0 < δ∧
    δ≤liminf (fun T : ℕ=> (T : ℝ)⁻¹ * ∑t ∈Finset.range T, infeasProb t) atTop

def LacksX₆ (staleProb : ℕ→ℝ) (tau : ℕ) : Prop :=
  ∃(ε δ: ℝ), 0 < ε∧ε≤1/2 ∧0 < δ∧
    (Nat.floor ((1 + ε) * tau) > tau) ∧
    ∀t : ℕ, tau < t →t ≤Nat.floor ((1 + ε) * tau) →staleProb t ≥1/2 + δ
```

Definition 2.11 (Possession — ordinary and robust)**.** Ordinary possession *of* X_i *is simply the negation of lacking it:* $\text{Possesses}X_i := \neg\,\text{Lacks}X_i$. Robust possession *is a strictly stronger, quantitative guarantee with its own shape for each* i *— for instance, robust* X_1 *asks for* $\liminf_t \text{mi}(t) > \frac{1}{2}$ *outright (not merely that the limsup fails to stay at or below some* $B < \frac{1}{2}$*), and robust* X_2 *asks for the danger probability to converge to* 0 *(not merely to fail to stay bounded away from* 0*).*

Lean 4 / Mathlib — `Phase0.lean, lines 110--156`

```
def PossessesX₁ (mi : ℕ→ℝ) : Prop := ¬LacksX₁ mi
def RobustX₁ (mi : ℕ→ℝ) : Prop := 1/2 < liminf (fun t => mi t) atTop

def PossessesX₂ (dangerProb : ℕ→ℝ) : Prop := ¬LacksX₂ dangerProb
def RobustX₂ (dangerProb : ℕ→ℝ) : Prop :=
  Tendsto dangerProb atTop (nhds 0)

def PossessesX₃ (cumBridge : ℕ→ℝ) : Prop := ¬LacksX₃ cumBridge
def RobustX₃ (cumBridge : ℕ→ℝ) : Prop :=
  Tendsto (fun T : ℕ=> cumBridge T / Real.sqrt T) atTop atTop

def PossessesX₄ (condEnt : ℕ→ℝ) : Prop := ¬LacksX₄ condEnt
def RobustX₄ (condEnt optProb : ℕ→ℝ) : Prop :=
  liminf (fun t : ℕ=> condEnt t) atTop = 0 ∧
  Tendsto optProb atTop (nhds 1)

def PossessesX₅ (infeasProb : ℕ→ℝ) : Prop := ¬LacksX₅ infeasProb
def RobustX₅ (infeasProb : ℕ→ℝ) : Prop :=
  Tendsto
```

```
    (fun T : ℕ=> (T : ℝ)⁻¹ * ∑t ∈Finset.range T, infeasProb t)
    atTop (nhds 0)

def PossessesX₆ (staleProb : ℕ→ℝ) (tau : ℕ) : Prop := ¬LacksX₆ staleProb tau
def RobustX₆ (staleProb : ℕ→ℝ)
             (tau : ℕ)
             (cleanSummary : ℕ→ℝ)
             (actualSummary : ℕ→ℝ) : Prop :=
  ¬LacksX₆ staleProb tau ∧
  ∀K : ℕ, cleanSummary K = actualSummary K
```

Remark 2.12 (The hierarchy is non-vacuous)**.** *Robust possession implies ordinary possession for each i, immediately from the definitions and the elementary fact* $\liminf \le \limsup$ *— e.g.* $\liminf_t \mathrm{mi}(t) > \frac{1}{2}$ *and* $\limsup_t \mathrm{mi}(t) \le B < \frac{1}{2}$ *cannot both hold, so robust X_1 rules out lacking X_1. This five-line observation is not separately named as a lemma in the repository (it is not needed as a hypothesis anywhere downstream); it is recorded here only so that the three-tier language — lacks / ordinarily possesses / robustly possesses — is visibly a genuine, non-collapsing hierarchy before it is used.*

Intuition — Investing

Ordinary possession is a low bar: it just means the desk hasn't been *caught* failing in the specific, worst-case way F_i describes. Robust possession is the bar a regulator or an LP would actually want met: not merely "has not yet blown up," but "demonstrably trending toward the safe regime, with a quantitative rate." Part I (Necessity) only ever needs the weak bar — lacking X_i is already enough to force linear regret. Part III (Sequential Dependence) needs the strong bar, because a chain of merely-not-yet-failing primitives gives no guarantee about what happens next.

3 The Environment Class $\mathcal{C}$

An environment belongs to $\mathcal{C}$ if it exhibits *at least one* of six structural failures. It need not exhibit all six, and it need not be adversarial in any other respect — $\mathcal{C}$ is defined by the presence of a single qualifying difficulty, not by general-purpose hardness. This is deliberate: the necessity results of Part I show that each structural failure, on its own, already forces failure on an algorithm lacking the matching primitive, so membership via any single property is enough to make that primitive indispensable.

3.1 The Six Structural Properties

Definition 3.1 (Structural properties P_1–P_6)**.**

P_1 ***Reward ambiguity.*** *There is a hidden parameter $\Theta^* \in \{0, 1\}$ and a state s at which the two possible environments disagree about which of two actions $a_0 \neq a_1$ is optimal.*

P_2 ***Absorbing traps.*** *There is a proper, nonempty set of states S_{abs} that, once entered, can never be left (every transition out of S_{abs} stays in S_{abs}), every reward earned inside it is ≤ 0, and it is reachable with positive probability from outside.*

P_3 ***Local optima.*** *There are disjoint nonempty sets of states S_{local}, S_{global} such that the myopically-best action at any state in S_{local} keeps you in S_{local}, some non-myopic action can reach S_{global} with positive probability, and the best reward obtainable in S_{local} is strictly below the best reward obtainable in S_{global}.*

P_4 ***Deterministic optimality.*** *There is a single optimal state-action pair (s^*, a^*) with a fixed positive reward gap Δ^* over every other action at s^*, together with a promise that s^* is visited with frequency at least $\nu^* > 0$.*

P_5 ***Constrained feasibility.*** *There is a proper, nonempty set of feasible actions F such that any action outside F, at any state, earns reward at most $-M$ for some fixed $M > 0$.*

P_6 ***Nonstationarity.*** *The environment is really a sequence of environments that differs, in reward or in transition dynamics at some state-action pair, between time τ and time $\tau + 1$.*

Lean 4 / Mathlib — `Phase0.lean, lines 167--213`

```
def HasP₁ (env₀ env₁ : Env S A 0) : Prop :=
  ∃(s : S) (a₀ a₁ : A), a₀ ≠a₁ ∧
    (∀ a : A, env₀.r (s, a₀) ≥env₀.r (s, a)) ∧
    (∀ a : A, env₁.r (s, a₁) ≥env₁.r (s, a))

def HasP₂ (env : Env S A 0) (S_abs : Set S) : Prop :=
  S_abs.Nonempty ∧S_abs ≠Set.univ ∧
  (∀ s ∈S_abs, ∀a : A, env.trans (s, a) S_abs = 1) ∧
  (∀ s ∈S_abs, ∀a : A, env.r (s, a) ≤0) ∧
  (∃ s ∉S_abs, ∃a : A, 0 < env.trans (s, a) S_abs)

def HasP₃ (env : Env S A 0) (S_local S_global : Set S) : Prop :=
  Disjoint S_local S_global ∧
  S_local.Nonempty ∧S_global.Nonempty ∧
  (∀ s ∈S_local, ∀a : A,
    (∀ a' : A, env.r (s, a) ≥env.r (s, a')) →
      env.trans (s, a) S_local = 1) ∧
  (∃ s ∈S_local, ∃a : A, env.trans (s, a) S_global > 0) ∧
  (∃ r_local r_global : ℝ,
    (∀ s ∈S_local, ∀a : A, env.r (s, a) ≤r_local) ∧
    (∀ s ∈S_global, ∃a : A, env.r (s, a) ≥r_global) ∧
    r_local < r_global)

def HasP₄ (env : Env S A 0) (s_star : S) (a_star : A)
    (visitFreq : Env S A 0 →S →ℝ→Prop) : Prop :=
  ∃(Δ_star : ℝ) (ν_star : ℝ),
    0 < Δ_star ∧0 < ν_star ∧
    (∀ a : A, a ≠a_star →
      env.r (s_star, a_star) - env.r (s_star, a) ≥Δ_star) ∧
    visitFreq env s_star ν_star

def HasP₅ (env : Env S A 0) (F : Set A) (M : ℝ) : Prop :=
  0 < M ∧F ⊂Set.univ ∧
  ∀(s : S) (a : A), a ∉F →env.r (s, a) ≤-M

def HasP₆ (envs : ℕ→Env S A 0) (tau : ℕ) : Prop :=
  ∃(s : S) (a : A),
    (envs tau).r (s, a) ≠(envs (tau + 1)).r (s, a) ∨
    (envs tau).trans (s, a) ≠(envs (tau + 1)).trans (s, a)
```

3.2 Class Membership

Definition 3.2 (Class $\mathcal{C}$)**.** *$\mathcal{E} \in \mathcal{C}$ if $\mathcal{E}$ exhibits at least one of P_1–P_6.*

Lean 4 / Mathlib — `Phase0.lean, lines 215--225`

```
def InClassC (env : Env S A 0)
    (visitFreq : Env S A 0 →S →ℝ→Prop) : Prop :=
```

```
(∃ env' : Env S A O, HasP₁ env env') ∨
(∃ S_abs : Set S, HasP₂ env S_abs) ∨
(∃ S_local S_global : Set S, HasP₃ env S_local S_global) ∨
(∃ (s_star : S) (a_star : A), HasP₄ env s_star a_star visitFreq) ∨
(∃ (F : Set A) (M : ℝ), HasP₅ env F M) ∨
(∃ (envs : ℕ→Env S A O) (tau : ℕ), envs 0 = env ∧HasP₆ envs tau)
```

3.3 Minimality

Each property above is stated with the fewest moving parts that still force the corresponding failure. P_1 needs only one ambiguous state and two candidate actions, not an arbitrary reward landscape; P_2 needs only that the trap be reachable and non-positive inside it, not that it be catastrophic; P_5 needs only one infeasible region with a uniform penalty, not a complex constraint set. This is a deliberate design choice: it makes the necessity results of Part I load-bearing for environments far more complicated than the ones actually used in the proofs.

Remark 3.3 (Why minimality matters for transfer)**.** *A necessity proof carried out on a stripped-down instance of a structural failure is* stronger*, not weaker, than one carried out on an elaborate instance, in the specific sense that matters here: every harder environment exhibiting the same qualitative structural failure — more states, more actions, a richer trap, a messier constraint set — contains the minimal instance as a special case of the difficulty an algorithm must overcome. An algorithm that cannot clear the minimal bar has no prospect of clearing a higher one. Conversely, this paper makes no claim that clearing the minimal bar is* sufficient *for harder instances of the same structural failure; sufficiency is handled separately, and only for the forward pipeline of Part III, under its own stated conditions. The point of minimality is one-directional: it is what licenses reading a necessity result proved here as a floor that persists into messier, real-world versions of the same underlying difficulty — which is exactly the property that makes the six primitives portable outside reinforcement learning rather than artifacts of one particular formalization.*

Intuition — Public Policy

A welfare program that cannot handle the simplest possible version of "some recipients will react to incentives in the intended way and some won't" has no chance with the real, messier population it will actually serve. Testing policy design against minimal, almost toy-like adversarial cases is not a weaker test than testing it against a fully realistic simulation — it is a more diagnostic one, because failure on the simple case cannot be blamed on incidental complexity.

4 From Formal Environments to Real Decision Structures

Section 1.3 previewed two claims and insisted they be kept apart. This section makes both precisely, in the order that matters: what kind of claim each one is, why the second is not debatable once its premise holds, and how to check the premise without fooling oneself in either direction.

4.1 Two Claims, Not One

Claim A (representation). A sequential decision process facing genuine uncertainty — an agent that acts, a world that responds stochastically, partial information about which state obtains, and a quantifiable consequence — can be formalized as an environment in the sense of Definition 2.1.

Claim B (inescapability). If an environment, however arrived at, instantiates one of the six structural properties of Definition 3.1, then the matching necessity result (Part I) applies to it with exactly the force it applies to the purpose-built environments of this paper.

	Structural Failure	Primitive forced	In one line
P_1	Reward ambiguity	X_1 — Objective Tracking	Tell apart two hypotheses that prescribe different actions.
P_2	Absorbing trap	X_2 — Cross-Context Safety Transfer	Stop touching the action that locks in permanent loss.
P_3	Local optimum	X_3 — Global Attractor Exploration	Keep probing past a comfortable plateau toward a better region.
P_4	Deterministic optimum	X_4 — Policy Simplification	Commit decisively once the evidence supports one option.
P_5	Constrained feasibility	X_5 — Feasibility Projection	Respect a hard constraint rather than trade it off.
P_6	Nonstationarity	X_6 — Feedback Adaptation	Discard stale evidence after the world changes.

Table 2: Each structural failure and the primitive Part I shows it forces. Names for X_1–X_6 are exactly the section names used in `Phase3.lean`.

Claim B is a theorem about a fixed formal object and needs no further argument beyond Part I itself: given the hypothesis, the conclusion follows, and nothing about *how* the hypothesis came to hold — which field, which vocabulary, which institution — enters the proof. Claim A is different: it asserts a correspondence between an open-ended class of real situations and a formal structure, and no proof assistant can check it, for the same reason none can check the Church–Turing thesis — "every real decision process under uncertainty" is not a mathematical object a quantifier can range over. This paper does not pretend otherwise, and offers instead the same kind of case such a thesis has always been argued on: convergent practice across independent fields, plus a theorem (von Neumann and Morgenstern) that closes the one gap in the representation that looks most like an arbitrary choice.

4.2 Why Realistic Decision Processes Generically Land in Class $\mathcal{C}$

The state/action/observation/transition/reward structure of Definition 2.1 is not a contribution of this paper: it is, in some equivalent form, the standard general-purpose formalism for sequential decision-making under uncertainty across robotics, operations research, control theory, and medicine, adopted in those fields precisely because so little else needs to be assumed (Kaelbling et al., 1998). The one piece that can look like an arbitrary modeling choice — why should "success" reduce to a single real number? — is not a choice at all for a rational decision-maker: von Neumann and Morgenstern (1944) prove that any preference ordering over uncertain outcomes satisfying four mild axioms (completeness, transitivity, continuity, and independence) is representable as maximizing the expectation of *some* real-valued utility function. (The independence axiom is the one most contested — the Allais paradox is the classical objection — so this is a conditional foundation, not an unconditional one; it is, however, the foundation nearly every quantitative field already builds on.)

Granting the representation, does a realistic process actually land in $\mathcal{C}$? Here the case is empirical, and economics has been making it for half a century under a different name. Arrow and Debreu (1954) prove that competitive equilibrium is optimal under an idealized complement of $\mathcal{C}$: complete information, no irreversible catastrophe, no coordination trap, full flexibility, a unique optimum, stationarity. Akerlof (1970) — work that earned a Nobel Memorial Prize shared with Spence and Stiglitz precisely for demonstrating this point generally — shows that ordinary markets (used cars, insurance, credit) violate the completeness of information, which is exactly P_1 under a different name. The macroeconomic literature on structural

breaks following the Lucas critique is P_6 under a different name; technology lock-in and poverty-trap models are P_3; systemic-risk and financial-contagion models are P_2. None of this paper's six structural failures needed to be invented; each already has a name and a literature in at least one field that is not reinforcement learning, which is itself evidence for Claim A independent of anything proved in this paper.

4.3 The Literature Tests One Failure at a Time

The case made so far is that real decision processes generically satisfy at least one of the six structural properties. A narrower, separate point follows: the formal apparatus built to make that case — one environment class unifying six independently-necessary structural properties, with every pairwise substitution between them ruled out and machine-checked — has no real precedent in the literature this paper draws on.

The standard practice in regret-lower-bound work is to isolate a single source of hardness. Lattimore and Szepesvári (2020), the standard reference for this methodology, builds essentially every lower bound in the bandit and reinforcement-learning literature around one hard instance engineered for one specific difficulty; combining several distinct hardness mechanisms into a single instance, with a separate necessity proof for each, is not how the technique is used there or, to this paper's knowledge, used anywhere else in that literature. Even where researchers have deliberately tried to test many capabilities at once, the same one-at-a-time discipline holds: Osband et al. (2020)'s bsuite, the most deliberately broad attempt at this in the empirical RL literature, organizes its environments so that each one isolates one or a small number of core capabilities, precisely so that failure on a given environment can be attributed to one specific deficiency rather than entangled with several. Where the literature has tried to combine even just two structural properties formally — safety and nonstationarity, the closest existing analogue to combining two of this paper's six — the result is not a unified solution but an open problem: a 2026 survey describes their combination as remaining "one of the most challenging topics of research" (Tomashevskiy, 2026), and recent empirical work attempting exactly that combination finds that current methods cannot satisfy both simultaneously (Coursey et al., 2026).

Against that backdrop, the claim this paper makes is precise rather than sweeping: six structural properties, each shown individually necessary in its own minimal environment (Part I), with every one of the thirty ordered pairs shown irreplaceable by the other (Part II), unified under one machine-checked formal apparatus. To this paper's knowledge, no prior formalization combines this many independently-proven-necessary structural properties into a single class with proven mutual independence across all of them. This is not a claim that six is exhaustive or that no harder combination could be built — only that the union itself, and the discipline of proving every pair of its components irreplaceable rather than assuming it, is new. The methodology used to build it — a minimal environment per property, a necessity proof, a home-turf independence check against every other property's canonical algorithm — does not stop at six; it is offered as a template for extending Class $\mathcal{C}$ with further structural properties under the same standard of proof.

This distinction matters beyond the count of properties combined. Section 1.2 already established that porting a single primitive proves nothing about the rest; concretely, verifying one client's objective-tracking, however rigorously, is verifying one fractured property of Class $\mathcal{C}$, not the environment the client is actually in — it implies nothing about whether the client will walk away from an unrecoverable mistake, keep exploring past a comfortable plateau, or adapt once the ground shifts. Only porting the union — all six primitives, with the independence and sequential-dependence proofs attached — licenses the inference this paper's cross-domain examples actually rely on.

4.4 The Disciplines This Required

Assembling Class $\mathcal{C}$ and the three results proved over it drew on several disciplines that do not, in ordinary practice, share a department, a conference, or a referee pool. Part I's necessity proofs are built from

measure-theoretic probability and the two-point/Le Cam and Fano techniques standard to sequential-decision lower bounds (Section 6). Part II's independence argument is a reduction, in the same spirit complexity theory uses the term: thirty pairwise hardness constructions collapse to one fixed-policy template checked against six characterizations Part I had already proved, rather than thirty separate direct arguments (Section 7). Part III's closing link rests on genuine martingale theory — an actual Doob convergence argument over a cycle-indexed filtration, not a restatement of Part I's techniques (Section 8.6). None of the above means anything without the discipline of formal verification itself: every definition and every theorem is machine-checked in Lean 4 over Mathlib, with zero `sorry`, zero custom axioms, zero `opaque` terms (`https://github.com/M-Ismail-ZA/IsmailsPrimitives`). And reading any of it outside reinforcement learning at all draws on the representation-theorem tradition of decision theory and economics — von Neumann and Morgenstern, Arrow and Debreu, Akerlof — not the bandit literature (Section 4.2).

None of the formal definitions underneath any of this carry domain vocabulary. `Env`, HasP_1–HasP_6, and InClassC (Definitions 2.1, 3.1, 3.2) are stated over abstract measurable state, action, and observation types and never mention reinforcement learning, economics, or any other field, by inspection. The cross-domain readings throughout this paper are therefore not adaptations of a reinforcement-learning result to new territory: the result was never reinforcement-learning-specific to begin with, so recognizing that a fund, a regulator, or a consultancy engagement instantiates HasP_2 is not porting a theorem across a boundary — the theorem was never stated on one side of a boundary in the first place.

That this particular combination had not been assembled before Section 4.3 checked for is not an accident of oversight: academic incentive structures reward depth within a single silo, not breadth across several. Promotion committees, conference referee pools, and one-to-three-year grant cycles all evaluate against a single subfield's norms, rewarding extension of an established paradigm over assembling one from parts with no shared home discipline — and none of this is a flaw; each is well suited to producing steady, reviewable progress within a field. None is suited to a project requiring simultaneous fluency in measure-theoretic probability, information theory, sequential-decision lower-bound construction, martingale convergence, and machine-checked formal verification, sustained across an arc with no publishable intermediate unit until independence and sequential dependence both close over all six primitives at once.

Freedom from institutional constraints is necessary for this kind of work and manifestly not sufficient for it — most work produced outside institutional settings stays within one silo too, by choice or convenience. What closes the gap is every discipline listed above present in one place, sustained long enough for all three parts to close together; that this combination is rare is a structural fact about how those disciplines are organized, not a claim this paper needs to make about the person who assembled it. This paper is offered as a case study in what crossing those silos deliberately can produce.

4.5 No Rhetoric Escapes a Genuine Instance

Granting Claim A's premise for a specific situation is a factual, falsifiable modeling question — exactly the kind Section 4.6 gives tools for, and exactly the kind one can get wrong in either direction. But once that question is answered — once a real process is shown, under any faithful formalization, to instantiate P_i — disputing the conclusion by redescribing the situation in different words changes nothing, for the same reason recompiling a halting program under a different variable-naming convention does not make it run forever. A fund that calls its absorbing trap a "concentrated high-conviction position" is still describing P_2. A committee that calls its failure to commit "maintaining strategic flexibility" is still describing P_4. The mathematics was never reading the label.

This cuts in both directions: a process that is *not* actually facing one of the six structural failures — genuine ambiguity resolved only by future evidence, a genuinely irreversible mistake, a genuinely reachable better region, evidence that has genuinely settled, a genuinely hard constraint, a rule that has genuinely changed — is not bound by the matching necessity result merely because it superficially resembles one. The power of a precise structural failure definition is exactly that it can be correctly judged absent, not only correctly judged present.

4.6 Recognizing the Six Structural Failures in Practice

Each test below is stated to be passed *or failed* by a real situation — the failure case is included deliberately, because a test nothing can fail is not a test. But these are diagnostic questions about one primitive at a time, and Class $\mathcal{C}$ is a union (Definition 3.2), not six independent checkboxes to clear in sequence and forget. A real decision process is not held fixed while one test is checked against it: the "not this" boundary for every test below implicitly assumes the reader's read of the situation (X_1) and the currency of that read (X_6) are already reliable. When they are not, a scenario that appears to clear one primitive's test can be exactly the visible symptom of a different, already-active failure — and left unaddressed, that failure does not sit inertly; it can compound into what a later primitive's test would flag, or into a trap no single test was checking for at all. This mirrors, informally, the same directed structure Part III proves formally in the other direction (Section 8): if possessing X_i is what creates the conditions to benefit from X_{i+1}, persistently failing to exercise X_i does not leave X_{i+1}'s test neutral either. This mirror claim is a practical corollary of taking Definition 3.2's union seriously across time, not itself one of Part III's machine-verified theorems — it is offered here as an instruction for how to use the six tests, not as a seventh one.

P_1 — **Reward ambiguity.**
At least two live courses of action; you cannot tell which is better today; only outcomes observed over time can. *Not this:* a choice between two options where one is already well-documented to dominate, provided that documentation is current — a dominance relationship that held once but has not been re-examined since is a live P_6 question wearing P_1's clean-pass language.

P_2 — **Absorbing trap.**
A specific action or pattern that, once taken, locks you into a state with no recovery path and no further upside. *Not this, cleanly:* a loss consistent with ordinary variance around a strategy that objective-tracking still supports — nothing about the read has changed, and next period's expected value is unchanged. *Not a clean pass, despite appearances:* taking the same loss and simply expecting the next cycle to recover it, without re-checking whether anything about the market or the strategy has actually changed. That expectation is not evidence against P_2; it is what a live P_6 failure — an unexamined belief carried forward unchanged — looks like from the inside, and repeating it is exactly how a run of ordinary-looking quarters can walk a desk into a state that functions as a trap, one small step at a time, with no single moment that looked like a door closing.

P_3 — **Local optimum.**
The best-looking option right now is not the option that leads to the best long-run outcome, and reaching the better outcome requires deliberately accepting worse results for a while. *Not this:* a free improvement available at no cost — if reaching the better region costs nothing, there is no trap to escape. Though an improvement that is genuinely free and stays untaken regardless is worth asking whether it has been recognized as better at all, which is P_1's question, not P_3's.

P_4 — **Commitment failure.**
The evidence already available decisively favors one option, and the choice is still being treated as open. *Not this:* an honest case where material new evidence is still arriving and could still overturn the leading option — premature commitment there would be the actual mistake.

P_5 — **Hard constraint.**
A specific region of actions is not merely worse but categorically off-limits, with no exchange rate against performance elsewhere. *Not this:* a soft preference, like favoring lower volatility, that trades off smoothly against other goals — P_5 requires a wall, not a preference. A constraint that was a wall until it became inconvenient, and was then redescribed as a preference, was never actually reclassified; it was violated.

P_6 — **Regime change.**
The underlying relationship being relied on has actually changed. *Not this:* a single surprising outcome fully consistent with an unchanged underlying process — ordinary noise is not a regime change, provided the same explanation is not being reached for every time. A pattern of surprises, each individually dismissed as noise, is no longer describable as a single surprising outcome.

A process can fail every test and sit safely in $\mathcal{C}^c$, the idealized complement — Section 4.2's claim is that this is the exception in practice, not that it is impossible in principle. But a single, one-time pass through all six tests is weaker evidence for that exception than it looks: because the tests are not independent of each other, a real process's best defense is not a clean audit taken once, but the same six primitives, actually possessed, rechecked as the cycle turns.

5 Standard Tools

Exactly one chain of general-purpose machinery is actually load-bearing across the rest of the paper: Shannon entropy and conditional entropy in their measure-theoretic form, and a Fano-type bound connecting conditional entropy to error probability. It is used in X_4's necessity proof (Section 6.4) and underlies the mutual information sequence used throughout Part III.

Definition 5.1 (Entropy and conditional entropy)**.** *For a finite-valued random variable X on a probability space and a sub-σ-algebra $\mathcal{F}$, the* conditional entropy $H(X \mid \mathcal{F}) := \int_\Omega \sum_x -\mathbb{P}[X{=}x \mid \mathcal{F}](\omega) \log \mathbb{P}[X{=}x \mid \mathcal{F}](\omega)\, d\mathbb{P}(\omega)$*, and the (unconditional)* entropy $H(X) := H(X \mid \bot)$.

Lean 4 / Mathlib — `Phase2CMI.lean, lines 24--34`

```
noncomputable def condEntropy
    {α : Type*} [Fintype α] [MeasurableSpace α] [MeasurableSingletonClass α]
    (X : Ω→α) (m : MeasurableSpace Ω) : ℝ:=
  ∫ω, ∑x : α,
    -(ℙ[(X ⁻¹' {x}).indicator (fun _ => (1 : ℝ)) | m] ω*
      Real.log (ℙ[(X ⁻¹' {x}).indicator (fun _ => (1 : ℝ)) | m] ω)) ∂ℙ

noncomputable def entropy
    {α : Type*} [Fintype α] [MeasurableSpace α] [MeasurableSingletonClass α]
    (X : Ω→α) : ℝ:=
  condEntropy X ⊥
```

Lemma 5.2 (Conditioning never increases entropy; determinism gives zero)**.** *$H(X \mid \mathcal{F}) \le H(X)$ for every $\mathcal{F}$. If $X = f \circ Y$ for a measurable f and $\mathcal{F}$ is at least as fine as Y's generated σ-algebra, then $H(X \mid \mathcal{F}) = 0$.*

Lean 4 / Mathlib — `Phase2CMI.lean, lines 158--173 (signatures)`

```
lemma condEntropy_le_entropy
    (X_rv : Ω→α) (hX : Measurable X_rv) (m : MeasurableSpace Ω) :
    condEntropy X_rv m ≤entropy X_rv

lemma condEntropy_of_deterministic
    (f : α→β) (hf : Measurable f) (X_rv : Ω→α) (hX : Measurable X_rv)
    (𝓕 : MeasurableSpace Ω) (h_le : MeasurableSpace.comap X_rv inferInstance ≤𝓕) :
    condEntropy (f ∘X_rv) 𝓕= 0
```

These two facts are exactly what makes every forward link in Part III collapse the conditional term to 0 via determinism, and what makes lim inf / lim sup bookkeeping throughout Part III well-posed.

Definition 5.3 (Binary entropy and its inverse). *$H(q) := -q\log q - (1-q)\log(1-q)$ for $q \in [0,1]$. On $(0, \frac{1}{2})$, H is a strictly increasing bijection onto $(0, \log 2)$; its inverse $H^{-1} : (0, \log 2) \to (0, \frac{1}{2})$ is the function used in Theorem 6.21.*

Lean 4 / Mathlib — `Phase1.lean, lines 215--221, 436--451`

```
noncomputable def binEntropy (q : ℝ) : ℝ:=
  -(q * Real.log q) - (1 - q) * Real.log (1 - q)

noncomputable def binEntropyInv (ε : ℝ) : ℝ:=
  if h : 0 < ε∧ε< Real.log 2 then
    Classical.choose (exists_binEntropy_eq h.1 h.2)
  else 0

theorem binEntropyInv_spec {ε : ℝ} (hε : 0 < ε) (hε2 : ε< Real.log 2) :
    0 < binEntropyInv ε∧binEntropyInv ε< 1 / 2 ∧
    binEntropy (binEntropyInv ε) = ε
```

Lemma 5.4 (Fano, conditional-entropy form). *Let A be a two-valued random variable, a^* a distinguished value, and X any random variable (representing whatever the predictor observes). If $H(A \mid X) \geq \varepsilon$ for some $\varepsilon \in (0, \log 2)$, then $\mathbb{P}(A \neq a^*) \geq H^{-1}(\varepsilon)$.*

Lean 4 / Mathlib — `Phase2CMI.lean, lines 974--982`

```
theorem fano_binary_error_lower_bound {ε : ℝ} (hε : 0 < ε) (hε2 : ε< Real.log 2)
    {A X : Type*} [MeasurableSpace A] [MeasurableSpace X]
    [Fintype A] [MeasurableSingletonClass A] [DecidableEq A]
    (hA_card : Fintype.card A = 2)
    (a_star : A) (A_rv : Ω→A) (X_rv : Ω→X)
    (hAmeas : Measurable A_rv) (hXmeas : Measurable X_rv)
    (hH_ge : ε≤condEntropy A_rv (MeasurableSpace.comap X_rv inferInstance)) :
    (ℙ {ω | A_rv ω≠a_star}).toReal ≥Phase1.binEntropyInv ε
```

Remark 5.5 (A second, independent entropy definition). *Phase 1 separately defines* condEntropyOf(μ, A, X) *(Phase1.lean, lines 51–57), used directly in X_4's discriminator (Definition 6.20). It is the same mathematical quantity as* condEntropy$(A, \text{comap}\, X)$ *above, packaged with an explicit measure argument rather than an ambient* `MeasureSpace` *instance; the two are not literally the same Lean definition, and neither is derived from the other in the repository.*

Remark 5.6 (Left for the bridge, not left over). *Fourteen declarations in Phase 1 are never invoked by any theorem in Parts I–III. They are not incidental — each sits adjacent to a specific obligation Part III leaves open (Section 8.1), positioned for whoever instantiates the chain rather than walked by this paper.*

Hypothesis testing. `klDiv`, `tvDist` *(Phase1.lean, lines 34, 37),* `pinsker_inequality` *(line 1454, bounding TV distance by $\sqrt{\text{KL}/2}$) — the bridge between Part III's native currency (entropy, mutual information) and Part I's (the TV-distance argument X_1 carries out directly).* `data_processing_tv` *(line 524) formalizes that passing two distributions through the same channel cannot manufacture distinguishability that was not already there — the tool that would let an implementer* prove *an* IsSufficientSummary *claim rather than assume it, which is what every theorem in Part III currently does.* `sum_error_ge_one_sub_tv` *(line 460) is the general-PMF version of the Le Cam step X_1's proof performs by hand for its one minimal witness — what generalizes X_1 beyond that witness.*

Bernoulli-KL calibration. `klBern`, `bernoulli_kl_lower_bound`, `bernoulli_kl_upper_bound` *(lines 1191, 1197, 1337), calibrated to precisely the $\frac{1}{2} \pm \Delta$ shape X_1's and X_6's environments already use — what concretizing the fully abstract* RobustX_4/RobustX_5 *propositions in IET 4 and IET 5 for an actual bandit-shaped algorithm would run through.*

Mixture and divergence. ***mixPMF***, ***JSD***, ***jsd_le_quarter_kl_sym*** *(lines 41, 1026, 1104).* JSD*'s range is* $[0, \log 2]$ *— exactly the ceiling every forward theorem in Part III converges toward — making it a candidate for what* MI_Summary_Seq*'s convergence claims would be derived from, rather than assumed.*

Martingale convergence. ***azuma_hoeffding*** *(line 576) upgrades IET 6's qualitative almost-sure martingale convergence (Theorem 8.10) to a quantitative rate.* ***kronecker_lemma*** *(line 880) is the classical bridge from a summable-series condition to Cesàro-average convergence —* RobustX_5 *is defined as exactly such an average (Definition 2.11) — connecting IET 6's martingale apparatus to an* X_5*-shaped averaged guarantee elsewhere in the chain.*

Every necessity, independence, and sequential-dependence result in this paper is proved using only the entropy/Fano chain of Section 5, plus elementary analysis local to each proof. The fourteen tools above were not needed to prove what this paper proves. They were needed, and left in place, for what this paper does not.

Intuition — Operations

Fano's inequality, in this form, says something almost mundane: if a predictor's residual uncertainty about a binary outcome — after seeing everything it's going to see — is still substantial, then it must sometimes guess wrong, at a rate the residual uncertainty pins down quantitatively. "Still genuinely unsure" and "sometimes wrong" are, quantitatively, the same fact viewed from two sides.

6 Part I — Ismail's Proof of Primitive Necessity

Each primitive is shown necessary by the same recipe. A minimal environment exhibiting one structural failure from Section 3 is exhibited; an algorithm is shown to suffer *linear* regret in that environment whenever it lacks the matching primitive (in the ordinary, weak sense of Definition 2.11 — the floor is proved against the weakest plausible failure, not against some artificially strong one). Linear regret is always enough to rule out sublinear regret, so no separate argument is needed for that direction. Six such results are proved; each is self-contained and uses only its own environment, never the other five.

6.1 Ismail's First Primitive — Objective Tracking (X_1)

Definition 6.1 (The two-armed environment pair)**.** *Fix a gap parameter* $\Delta \in (0, \frac{1}{2})$*. Two stateless, two-armed bandit environments are defined: in* $\mathcal{E}_1^{(0)}$*, arm* 0 *pays* $\frac{1}{2} + \Delta$ *and arm* 1 *pays* $\frac{1}{2} - \Delta$ *(as deterministic reward; the algorithm's only signal is a noisy* Bernoulli *observation with the corresponding mean); in* $\mathcal{E}_1^{(1)}$ *the two arms are swapped. The two environments agree on everything except which arm is better.*

Lean 4 / Mathlib — `Phase2.lean, lines 1702--1730`

```
structure BanditParam where
  Δ: ℝ
  hΔ0 : 0 < Δ
  hΔ1 : Δ< 1/2

noncomputable def env₁_0 (bp : BanditParam) : SixPrimitives.Env Unit (Fin 2) Bool where
  trans := Kernel.const _ (Measure.dirac ())
  obs := { toFun := fun (_, a) =>
              if a = (0 : Fin 2)
              then bernoulliMeasure (1/2 + bp.Δ) (by linarith [bp.hΔ0]) (by linarith [bp.h
                  Δ1])
              else bernoulliMeasure (1/2 - bp.Δ) (by linarith [bp.hΔ1]) (by linarith [bp.h
                  Δ0])
           measurable' := measurable_of_finite _ }
  r := fun (_, a) => if a = 0 then 1/2 + bp.Δ else 1/2 - bp.Δ
```

```
  hr_meas := measurable_of_finite _
  μ₀:= Measure.dirac ()
  hμ₀ := inferInstance

noncomputable def env₁_1 (bp : BanditParam) : SixPrimitives.Env Unit (Fin 2) Bool where
  trans := Kernel.const _ (Measure.dirac ())
  obs := { toFun := fun (_, a) =>
              if a = (0 : Fin 2)
              then bernoulliMeasure (1/2 - bp.Δ) (by linarith [bp.hΔ1]) (by linarith [bp.h
                  Δ0])
              else bernoulliMeasure (1/2 + bp.Δ) (by linarith [bp.hΔ0]) (by linarith [bp.h
                  Δ1])
           measurable' := measurable_of_finite _ }
  r := fun (_, a) => if a = 0 then 1/2 - bp.Δ else 1/2 + bp.Δ
  hr_meas := measurable_of_finite _
  μ₀:= Measure.dirac ()
  hμ₀ := inferInstance
```

Remark 6.2 (Class membership)**.** $\mathrm{HasP}_1(\mathcal{E}_1^{(0)}, \mathcal{E}_1^{(1)})$ *holds with the witnessing state being the unique state of* `Unit` *and the two candidate actions* $0 \neq 1$*: arm* 0 *is optimal in* $\mathcal{E}_1^{(0)}$ *and arm* 1 *is optimal in* $\mathcal{E}_1^{(1)}$*. This is verified directly (*`env₁_0_hasP₁`*, Phase2.lean lines 1731–1738) and reproduced in Appendix B.*

Definition 6.3 (The X_1 discriminator)**.** *For an algorithm* alg*, let* $\mathrm{mi}(t)$ *be the total variation distance between the time-*t *action distributions induced by running* alg *in* $\mathcal{E}_1^{(0)}$ *and in* $\mathcal{E}_1^{(1)}$ *respectively.* alg *lacks* X_1 *if* $\limsup_t \mathrm{mi}(t) \leq B$ *for some* $B < \frac{1}{2}$ *(Definition 2.10,* F_1*).*

Remark 6.4 (A naming note)**.** *Phase 0 glosses* F_1 *as "bounded mutual information," and the sequence is named* `miSeq` *throughout the repository. The concrete witness instantiated here is a total-variation distance between action marginals, not a Shannon mutual information. The two are playing the same conceptual role — both measure how distinguishable two hypotheses are to an observer of the algorithm's actions, and both appear in essentially the same place in a Le Cam-style two-point lower bound — but they are not the same quantity, and this paper states precisely which one is proved.*

Lean 4 / Mathlib — `Phase3.lean, lines 88--97`

```
noncomputable def miSeq [Fintype A] [MeasurableSingletonClass A]
    (env₀ env₁ : SixPrimitives.Env S A O)
    (alg : SixPrimitives.Algorithm A O Sig)
    (hEnv₀ : Phase2.EnvIsMarkov env₀)
    (hEnv₁ : Phase2.EnvIsMarkov env₁)
    (hAlg : Phase2.AlgIsMarkov alg)
    (t : ℕ) : ℝ:=
  tvDistMeasure
    (Phase2.actionMarginal env₀ alg env₀.hr_meas hEnv₀ hAlg (t + 1) ⟨t, Nat.lt_succ_self t⟩)
    (Phase2.actionMarginal env₁ alg env₁.hr_meas hEnv₁ hAlg (t + 1) ⟨t, Nat.lt_succ_self t⟩)
```

Theorem 6.5 (Necessity of X_1)**.** *For any algorithm* alg *(with Markov kernels) lacking* X_1 *relative to* $(\mathcal{E}_1^{(0)}, \mathcal{E}_1^{(1)})$*, there is a constant* $c > 0$ *such that, eventually,*

$$c \cdot T \;\leq\; \mathrm{regret}(\mathcal{E}_1^{(0)}, \mathrm{alg}, T) + \mathrm{regret}(\mathcal{E}_1^{(1)}, \mathrm{alg}, T).$$

Proof sketch. Fix $B < \frac{1}{2}$ witnessing F_1 and $\eta > 0$ with $B + \eta < \frac{1}{2}$; since $\mathrm{mi}(t) \in [0, 1]$ always, $\limsup \mathrm{mi} \leq B$ gives $\mathrm{mi}(t) < B + \eta$ eventually, say for $t \geq t_0$. A direct computation shows the per-step reward gap in $\mathcal{E}_1^{(0)}$

at step t is $2\Delta \cdot \mathbb{P}_0(A_t = 1)$, and symmetrically the gap in $\mathcal{E}_1^{(1)}$ is $2\Delta \cdot \mathbb{P}_1(A_t = 0)$, where $\mathbb{P}_i$ denotes the law of the action at step t under environment $\mathcal{E}_1^{(i)}$. Since actions are binary, the total variation distance between the two action laws equals $|\mathbb{P}_0(A_t = 0) - \mathbb{P}_1(A_t = 0)|$, so

$$\mathbb{P}_0(A_t = 1) + \mathbb{P}_1(A_t = 0) = 1 - \big(\mathbb{P}_0(A_t = 0) - \mathbb{P}_1(A_t = 0)\big) \;\geq\; 1 - \mathrm{mi}(t) \;>\; 1 - (B + \eta) \;>\; \tfrac{1}{2}$$

for $t \geq t_0$ — the classical Le Cam two-point bound. Summing the two per-step gaps over $t \geq t_0$ and absorbing the bounded contribution of the first t_0 steps into a slightly smaller constant gives the stated linear bound, with c any positive number below $2\Delta(1 - (B + \eta))$. □

Intuition — Investing

Picture two possible worlds that agree on everything except which of two strategies is the winning one, and no other signal in the data distinguishes them faster than the algorithm's own trades do. If the algorithm's trading pattern looks statistically similar across both worlds — it isn't tilting its book differently depending on which world it's actually in — then in at least one of the two worlds it is persistently overweighting the losing strategy. The size of that persistent error, multiplied by the gap between the two strategies' returns, is a drag that accumulates linearly, forever, no matter how much capital or time the desk has.

Remark 6.6 (Full proof)**.** *The theorem statement above is exact; the displayed proof is a faithful sketch of the argument's shape. The complete machine-checked proof (`necessity_x1`, Phase3.lean lines 122–561, some 440 lines of measure-theoretic bookkeeping carrying out exactly the steps above) is available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (see also Appendix A). The proposition itself, not merely the proof technique, is standard: this exact two-point construction is presented as a worked example in Lattimore and Szepesvári (2020).*

6.2 Ismail's Second Primitive — Cross-Context Safety Transfer (X_2)

Definition 6.7 (The corridor-and-trap environment)**.** *Fix $K \in \mathbb{N}_{>0}$. The environment $\mathcal{E}_2(K)$ has states $\{0, 1, \ldots, K-1, K, K+1\}$. From any corridor state $s < K$: action 0 ("safe") earns reward 1 and advances to $s+1$; action 1 ("danger") earns reward 0 and jumps straight to state $K+1$. State K is a safe harbor — every action keeps you there, earning reward 1 forever. State $K+1$ is the trap — every action keeps you there, earning reward 0 forever. Observations reveal the state exactly.*

Lean 4 / Mathlib — `Phase2.lean, lines 1934--1953`

```
noncomputable def env₂ (K : ℕ) (hK : 0 < K) :
    SixPrimitives.Env (Fin (K + 2)) (Fin 2) (Fin (K + 2)) where
  trans := Kernel.deterministic
    (fun (p : Fin (K + 2) ×Fin 2) =>
      if h : p.1.val < K then
        if p.2 = 0 then ⟨p.1.val + 1, by omega⟩ else ⟨K + 1, by omega⟩
      else p.1)
    (by measurability)
  obs := Kernel.deterministic
    (fun (p : Fin (K + 2) ×Fin 2) =>
      if h : p.1.val < K then
        if p.2 = 0 then ⟨p.1.val + 1, by omega⟩ else ⟨K + 1, by omega⟩
      else p.1)
    (by measurability)
  r := fun (s, a) =>
    if s.val < K then (if a = 0 then 1 else 0) else if s.val = K then 1 else 0
  hr_meas := measurable_of_finite _
  μ₀:= Measure.dirac ⟨0, by omega⟩
```

```
  hμ₀ := inferInstance

def env₂_trap (K : ℕ) : Set (Fin (K + 2)) := {⟨K + 1, by omega⟩}
```

Remark 6.8 (Class membership and the optimal value)**.** $\mathrm{HasP}_2(\mathcal{E}_2(K), \{K{+}1\})$ *is verified directly (*`env₂_hasP₂`*, Phase2.lean lines 1956–1974; reproduced in Appendix B): the trap is nonempty, proper, absorbing, earns reward* ≤ 0*, and is reachable with positive probability from state* 0*. Since always playing "safe" earns reward* 1 *at every single step with certainty,* $\mathrm{optValue}(\mathcal{E}_2(K), T) = T$ *exactly — regret in this environment is simply* $T - \mathrm{algValue}$*.*

Definition 6.9 (Conditional danger probability)**.** *Let* S_k *be the event of having never visited the trap by step* k*. The* conditional danger probability *at step* k *is* $\mathbb{P}(\textit{danger action at } k \mid S_k)$ *— the probability of gambling on the dangerous action at step* k*, given survival so far (defined as* 0 *if* $\mathbb{P}(S_k) = 0$*).*

Lean 4 / Mathlib — `Phase2.lean, lines 4806--4838`

```
def survivedToStep (K : ℕ) (hK : 0 < K) (k : ℕ) :
    Set (Phase2.Trajectory (Fin 2) (Fin (K+2)) (k+1)) :=
  let trap := Fin.last (K+1)
  { ω| Phase2.state_t (Phase2.env₂ K hK) (Phase2.env₂_isDet K hK).toTrans ωk ≠trap }

noncomputable def survivalProbSeq
    (K : ℕ) (hK : 0 < K)
    (alg : SixPrimitives.Algorithm (Fin 2) (Fin (K+2)) Sig)
    (_hAlg : Phase2.AlgIsMarkov alg)
    (k : ℕ) : ℝ:=
  ((Phase2.trajMeasure (Phase2.env₂ K hK) alg (Phase2.env₂ K hK).hr_meas (k+1))
    (survivedToStep K hK k)).toReal

noncomputable def dangerAndSurvivalProbSeq
    (K : ℕ) (hK : 0 < K)
    (alg : SixPrimitives.Algorithm (Fin 2) (Fin (K+2)) Sig)
    (_hAlg : Phase2.AlgIsMarkov alg)
    (aDanger : Fin (K+2) →Fin 2)
    (k : ℕ) : ℝ:=
  let surv_set := survivedToStep K hK k
  let danger_set := { ω| (ω ⟨k, Nat.lt_succ_self k⟩).1 =
    aDanger (Phase2.state_t (Phase2.env₂ K hK) (Phase2.env₂_isDet K hK).toTrans ωk) }
  ((Phase2.trajMeasure (Phase2.env₂ K hK) alg (Phase2.env₂ K hK).hr_meas (k+1))
    (surv_set ∩danger_set)).toReal

noncomputable def condDangerProbSeq
    (K : ℕ) (hK : 0 < K)
    (alg : SixPrimitives.Algorithm (Fin 2) (Fin (K+2)) Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (aDanger : Fin (K+2) →Fin 2)
    (k : ℕ) : ℝ:=
  let surv := survivalProbSeq K hK alg hAlg k
  if surv = 0 then 0 else dangerAndSurvivalProbSeq K hK alg hAlg aDanger k / surv
```

Remark 6.10 (A naming note: not literally LacksX_2)**.** *Definition 2.10's* F_2 *is stated for the* unconditional *marginal danger probability. In a trap environment that condition is the wrong one to ask for: once a positive fraction of trajectories have already been absorbed, the unconditional probability of seeing the danger action again decays toward* 0 *regardless of how recklessly the surviving trajectories behave, making*

an unconditional bound vacuous as a description of persistent risk-taking. The theorem below is therefore stated directly against the conditional probability above, not against $\mathrm{Lacks}X_2$ *as literally defined in Phase 0 — a deliberate, environment-specific strengthening in the same spirit as* F_2*, not an instance of it.*

Theorem 6.11 (Necessity of X_2)**.** *Fix* $\delta \in (0, 1]$*. If, for every* k*, the algorithm's conditional danger probability at step* k *is at least* δ*, then* alg *has linear regret in* $\mathcal{E}_2(K)$*, with regret eventually at least* $\big(1 - (1-\delta)^K\big)(T - K)$*.*

Lean 4 / Mathlib — `Phase3.lean, lines 931--940`

```
theorem necessity_x2
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
    (K : ℕ) (hK : 0 < K)
    (alg : SixPrimitives.Algorithm (Fin 2) (Fin (K + 2)) Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (δ : ℝ) (hδ_pos : 0 < δ) (hδ_le_one : δ≤1)
    (h_cond : ∀k,
       Phase2.condDangerProbSeq K hK alg hAlg (env₂_dangerAction K) k ≥δ) :
    LinearRegret (Phase2.env₂ K hK) alg
```

Proof sketch. Surviving K corridor steps requires avoiding the δ-probability danger event at each one, conditional on having survived so far; this forces $\mathrm{survivalProbSeq}(K) \le (1-\delta)^K$ by a direct induction on the conditional probabilities. If the algorithm is trapped by step K, a potential-function argument bounds its total reward through any horizon $T \ge K$ by K outright (every step thereafter earns 0); if it survives, its reward through T is at most T trivially. Splitting the expectation over the trapped and surviving events gives $\mathrm{algValue}(\mathcal{E}_2(K), \mathrm{alg}, T) \le T \cdot \mathrm{survivalProbSeq}(K) + K \cdot (1 - \mathrm{survivalProbSeq}(K))$, and since $\mathrm{optValue} = T$,

$$\mathrm{regret}(T) \;\ge\; (T-K)\big(1 - \mathrm{survivalProbSeq}(K)\big) \;\ge\; (T-K)\big(1-(1-\delta)^K\big),$$

which is linear in T since $1 - (1-\delta)^K > 0$ whenever $\delta > 0$. □

Remark 6.12 (Full proof and a parametric generalization)**.** *The complete proof (`necessity_x2`, Phase3.lean lines 931–1222) is available at* https://github.com/M-Ismail-ZA/IsmailsPrimitives *(Appendix A). A second theorem,* `necessity_x2_parametric` *(lines 1223–1523), proves the same bound for any danger-action labelling that agrees with* `env₂_dangerAction` *on the corridor states; it is used in Part II to test other algorithms against* $\mathcal{E}_2(K)$ *without committing to one fixed labelling at the trap state. The general difficulty of catastrophe-avoidance is an active research area; the closest analogue is Plaut et al. (2025), though its mechanism (querying a mentor) differs from the fixed-corridor construction used here.*

Intuition — Operations

A single irreversible mistake — the regulatory violation that triggers a license revocation, the safety bypass that causes a fatal accident, the leverage call that wipes the account — caps everything that follows, no matter how well things had been going up to that point. The relevant question is never "how often does this go wrong on average," which can look small even as the firm slides toward the trap; it is "how often does this go wrong, given that it hasn't gone wrong yet." An operation that gambles at a steady conditional rate, survival after survival, is on a clock — the probability of eventually hitting the trap converges to certainty, and the regret of having built on top of that gamble grows without bound.

6.3 Ismail's Third Primitive — Global Attractor Exploration (X_3)

Definition 6.13 (The local-optimum/bridge environment)**.** *Fix a bridge-success probability $p \in [0,1]$. The environment $\mathcal{E}_3(p)$ has two states: 0 ("local") and 1 ("global"). State 1 is absorbing and pays reward 1 forever, regardless of action. From state 0: action 0 ("stay") deterministically stays at 0 and pays $\frac{1}{2}$; action 1 ("bridge") pays 0 this step and moves to state 1 with probability p, otherwise stays at 0.*

Lean 4 / Mathlib — `Phase2.lean, lines 2095--2112`

```
private noncomputable def env₃_trans (p : ℝ) (hp0 : 0 ≤p) (hp1 : p ≤1) :
    Kernel (Fin 2 ×Fin 2) (Fin 2) where
  toFun := fun sa =>
    if sa.1 = 1 then Measure.dirac 1
    else if sa.2 = 0 then Measure.dirac 0
    else (PMF.bernoulli ⟨p, hp0⟩
          (show (⟨p, hp0⟩ : ℝ≥0) ≤1 from by exact_mod_cast hp1)).toMeasure.map
          (fun b : Bool => if b then (1 : Fin 2) else 0)
  measurable' := measurable_of_countable _

noncomputable def env₃ (p : ℝ) (hp0 : 0 ≤p) (hp1 : p ≤1) :
    SixPrimitives.Env (Fin 2) (Fin 2) Unit where
  trans := env₃_trans p hp0 hp1
  obs := Kernel.const _ (Measure.dirac ())
  r := fun (s, a) => if s = (1 : Fin 2) then 1 else if a = (0 : Fin 2) then 1/2 else 0
  hr_meas := measurable_of_finite _
  μ₀:= Measure.dirac 0
  hμ₀ := inferInstance
```

Remark 6.14 (Class membership, and a calibration)**.** $\mathrm{HasP}_3(\mathcal{E}_3(p), \{0\}, \{1\})$ *is verified directly (`env₃_hasP₃`, Phase2.lean lines 2114–2159; reproduced in Appendix B): staying greedy at the local state never leaves it, the bridge reaches the global state with positive probability, and the global state's reward strictly exceeds the local one. Unlike $\mathcal{E}_1, \mathcal{E}_2$, the instance used for a given horizon T is calibrated to that horizon: the theorem below uses $p = 1/\sqrt{T}$, so a* different *member of the $\mathcal{E}_3(p)$ family witnesses necessity at each horizon, rather than one fixed environment analyzed as T grows.*

Theorem 6.15 (Necessity of X_3)**.** *Fix $C_{\mathrm{tot}} > 0$. For every horizon $T \geq 4C_{\mathrm{tot}}^2$, if an algorithm's cumulative bridge-action count never exceeds C_{tot} at any horizon (in the instance $\mathcal{E}_3(1/\sqrt{T})$), then*

$$\mathrm{regret}\big(\mathcal{E}_3(1/\sqrt{T}), \mathrm{alg}, T\big) \;\geq\; \frac{T}{2} - (1 + C_{\mathrm{tot}})\sqrt{T}.$$

Lean 4 / Mathlib — `Phase3.lean, lines 1556--1577`

```
theorem necessity_x3
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
    (C_tot : ℝ) (hC_tot_pos : 0 < C_tot)
    (T : ℕ) (hT_large : (4 * C_tot ^ 2 : ℝ) ≤(T : ℝ))
    (alg : SixPrimitives.Algorithm (Fin 2) Unit Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (h_bound : ∀(T' : ℕ), Phase2.bridgeCumSeq
               (Phase2.env₃ (1 / Real.sqrt (T : ℝ))
                 (p_nonneg C_tot hC_tot_pos T hT_large)
                 (p_le_one C_tot hC_tot_pos T hT_large))
               alg
               (Phase2.env₃ (1 / Real.sqrt (T : ℝ))
```

```
              (p_nonneg C_tot hC_tot_pos T hT_large)
              (p_le_one C_tot hC_tot_pos T hT_large)).hr_meas
            1 T' ≤C_tot) :
  (T : ℝ) / 2 - (1 + C_tot) * Real.sqrt (T : ℝ) ≤
  SixPrimitives.regret
    (Phase2.env₃ (1 / Real.sqrt (T : ℝ))
      (p_nonneg C_tot hC_tot_pos T hT_large)
      (p_le_one C_tot hC_tot_pos T hT_large))
    alg T
```

Remark 6.16 (This is exactly LacksX_3)**.** *Unlike X_2's necessity proof, the hypothesis here — a uniform bound C_{tot} on the cumulative bridge count at every horizon — is precisely Definition 2.10's F_3, just written out rather than wrapped in the name* LacksX_3*. No environment-specific refinement was needed in this case.*

Proof sketch. Write $p = 1/\sqrt{T}$. Reaching the global state at all requires at least one of at most C_{tot} bridge attempts to succeed, each independently with probability p; a union bound caps the probability of ever reaching it by $C_{\text{tot}} \cdot p$, and reward exceeds $\frac{1}{2}$ only after that happens, capping the algorithm's value at roughly algValue $\leq T/2 + T \cdot (p \cdot C_{\text{tot}}) = T/2 + C_{\text{tot}}\sqrt{T}$. Meanwhile the reference algorithm that always attempts the bridge succeeds, in expectation, within $1/p = \sqrt{T}$ attempts and then collects reward 1 for essentially the rest of the horizon, giving optValue $\geq T - \sqrt{T}$. Subtracting gives the stated bound. □

Remark 6.17 (Full proof)**.** *The complete proof (`necessity_x3`, Phase3.lean lines 1556–1633, with two short supporting lemmas at lines 1531–1552) is available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (Appendix A). The environment is the minimal case of a long-standing benchmark family for exactly this phenomenon — local-optimum entrapment resisting costly exploration — introduced as RiverSwim by Strehl and Littman (2008) and used since as the Chain MDP by Osband et al. (2016), among many others.*

Intuition — Investing

A fund sitting on a comfortable, reliably mediocre strategy earning "half of what's possible" will never discover the better regime if it only ever allocates a small, fixed, one-off research budget to exploring it — the discovery either happens early, by luck, within that fixed budget, or it never happens, and the fund spends the entire remaining horizon stuck at half-value. Larger fixed budgets help, but only up to a fixed cap; the cap has to keep *growing with the time horizon* faster than $\sqrt{T}$ for genuine discovery to become likely as the fund operates longer. A one-time pilot program is not the same commitment as an exploration budget that scales with how long the firm has been running.

6.4 Ismail's Fourth Primitive — Policy Simplification (X_4)

Definition 6.18 (The single-state, single-best-action environment)**.** *$\mathcal{E}_4$ has one state, two actions, no transitions to track: action 0 always pays reward 1, action 1 always pays reward 0.*

Lean 4 / Mathlib — `Phase2.lean, lines 3742--3748`

```
noncomputable def env₄ : SixPrimitives.Env Unit (Fin 2) Unit where
  trans := Kernel.const _ (Measure.dirac ())
  obs := Kernel.const _ (Measure.dirac ())
  r := fun (_, a) => if a = 0 then 1 else 0
  hr_meas := measurable_of_finite _
  μ₀ := Measure.dirac ()
  hμ₀ := inferInstance
```

Remark 6.19 (Class membership). *This is the minimal possible instance of P_4: one state, a reward gap of exactly $\Delta^* = 1$, and the unique state is visited with frequency $\nu^* = 1$ trivially.* $\mathrm{HasP}_4(\mathcal{E}_4, (), 0)$ *is verified directly (`env₄_hasP₄`, Phase2.lean lines 3760–3765; reproduced in Appendix B).*

The discriminating quantity for X_4 is genuinely information-theoretic — unlike X_1's total-variation surrogate, this is the algorithm's own conditional entropy about its next action, given its own history.

Definition 6.20 (The X_4 discriminator). $\mathrm{condEnt}(t) := H(A_t \mid \mathrm{history}_t)$, *the entropy of the action taken at step t conditional on the full trajectory prefix up to t, under the law induced by running* alg *in $\mathcal{E}_4$.* alg *lacks X_4 if* $\liminf_t \mathrm{condEnt}(t) \geq \varepsilon$ *for some fixed $\varepsilon > 0$ (Definition 2.10, F_4) — informally, if the algorithm never stops being genuinely unsure, conditional on everything it has seen, about which action to take.*

Lean 4 / Mathlib — Phase3.lean, lines 1653--1660

```
noncomputable def condEntSeqX4
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    (alg : SixPrimitives.Algorithm (Fin 2) Unit Sig)
    (t : ℕ) : ℝ:=
  Phase1.condEntropyOf
    (Phase2.trajMeasure Phase2.env₄ alg Phase2.env₄.hr_meas (t + 1))
    (Phase2.traj_action t (Nat.lt_succ_self t))
    (traj_prefix t (Nat.lt_succ_self t))
```

Theorem 6.21 (Necessity of X_4, explicit form). *Fix $0 < \eta < \varepsilon < \log 2$. If* $\mathrm{condEnt}(t) \geq \varepsilon$ *eventually, then there is a horizon t_0 beyond which*

$$H^{-1}(\varepsilon - \eta) \cdot (T - t_0) \;\leq\; \mathrm{regret}(\mathcal{E}_4, \mathrm{alg}, T) \qquad \textit{eventually in } T,$$

where H^{-1} is the functional inverse of the binary entropy function on $[0, \log 2]$.

Lean 4 / Mathlib — Phase3.lean, lines 1662--1671

```
theorem necessity_x4_explicit_bound
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
    (ε : ℝ) (_hε : 0 < ε) (hε2 : ε< Real.log 2)
    (alg : SixPrimitives.Algorithm (Fin 2) Unit Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (hLacks_eps : ε≤liminf (condEntSeqX4 alg) atTop)
    (η : ℝ) (hη_pos : 0 < η) (hη_lt : η< ε) :
    ∃t₀ : ℕ, ∀ᶠT : ℕin atTop,
      Phase1.binEntropyInv (ε - η) * ((T : ℝ) - t₀) ≤regret Phase2.env₄ alg T
```

Proof sketch. Eventually $\mathrm{condEnt}(t) \geq \varepsilon - \eta$. Fano's inequality in its conditional-mutual-information form (Section 5) converts a conditional-entropy floor on a binary random variable into a floor on the probability of that variable taking its non-distinguished value: here, $\mathbb{P}(A_t \neq 0 \mid \mathrm{history}) \geq H^{-1}(\varepsilon - \eta)$ at every such t. Since action 1 costs exactly one unit of reward relative to action 0 in $\mathcal{E}_4$, summing this per-step error probability over $t \geq t_0$ gives the stated linear floor on regret directly. □

Theorem 6.22 (Necessity of X_4). *Any algorithm lacking X_4 in $\mathcal{E}_4$ has linear regret in $\mathcal{E}_4$.*

Lean 4 / Mathlib — Phase3.lean, lines 1831--1837

```
theorem necessity_x4
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
```

```
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
    (alg : SixPrimitives.Algorithm (Fin 2) Unit Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (hLacks : LacksX₄ (condEntSeqX4 alg)) :
    LinearRegret Phase2.env₄ alg
```

Remark 6.23 (Full proof). *necessity_x4 (Phase3.lean lines 1831–1857) is a direct corollary of necessity_x4_explicit_bound: unfold* $\mathrm{Lacks}X_4$ *to get* ε*, shrink it slightly to stay clear of* $\log 2$*, and apply the explicit theorem. Both complete proofs, together with the supporting Fano machinery from Phase2CMI, are available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (Appendix A). The general link between residual entropy over an optimal action and regret is Russo and Van Roy (2016)'s information-theoretic framework; the specific zero-uncertainty construction used here does not appear to be pre-stated in that or related work.*

Intuition — Consulting

This is the only one of the six environments with no real dynamics at all — one state, one obviously correct call, forever. Failing here isn't about facing a hard problem; it is about a decision process that keeps re-litigating an already-settled question. A committee that, after years of the same evidence arriving every quarter, still spends real deliberation time genuinely torn on a question with one clearly correct answer is not being careful — the carefulness was appropriate once, before the evidence came in. Continued indecision after the evidence is no longer prudence; each quarter spent still "deciding" is a quarter paying the cost of the wrong answer half the time.

6.5 Ismail's Fifth Primitive — Feasibility Projection (X_5)

Definition 6.24 (The forbidden-band environment). *$\mathcal{E}_5$ has a continuous action space $A = \mathbb{R}$ and a single state. The feasible set is $F = [0, \frac{1}{3}] \cup [\frac{2}{3}, 1]$. A feasible action a earns reward $\max\big(0,\, 1 - 3\min(|a|, |1-a|)\big)$ — which peaks at 1 exactly at $a = 0$ or $a = 1$ and falls linearly to 0 at the edges of F nearest the forbidden middle band. Any infeasible action (the open band $(\frac{1}{3}, \frac{2}{3})$, or anything outside $[0, 1]$) earns a flat penalty of -1.*

Lean 4 / Mathlib — Phase2.lean, lines 3838–3847

```
def feasSet₅ : Set ℝ := Set.Icc 0 (1/3) ∪ Set.Icc (2/3) 1
def penalty₅ : ℝ := 1

noncomputable def env₅ : SixPrimitives.Env Unit ℝ Unit where
  r := fun (_, a) =>
    if a ∈ feasSet₅ then max 0 (1 - 3 * min (|a|) (|1 - a|)) else -penalty₅
```

Remark 6.25 (Class membership and optimal value). $\mathrm{HasP}_5(\mathcal{E}_5, F, 1)$ *is verified directly (env₅_hasP₅, Phase2.lean lines 3862–3868; reproduced in Appendix B): F is a proper subset of $\mathbb{R}$ and every infeasible action earns exactly -1. Always playing $a = 0$ is feasible and earns reward 1 every step, so* $\mathrm{optValue}(\mathcal{E}_5, T) = T$ *(env₅_optValue, Phase2.lean line 3898).*

Theorem 6.26 (Necessity of X_5). *Any algorithm lacking X_5 in $\mathcal{E}_5$ has linear regret in $\mathcal{E}_5$.*

Lean 4 / Mathlib — Phase3.lean, lines 2046–2053

```
theorem necessity_x5
    {Sig : Type*} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
```

```
    (alg : SixPrimitives.Algorithm ℝUnit Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (hLacks : LacksX₅ (Phase2.infeasProbSeq
       Phase2.env₅ alg Phase2.env₅.hr_meas infeasSet₅)) :
    LinearRegret Phase2.env₅ alg
```

Proof sketch. Unfolding LacksX_5 gives $\delta > 0$ with the Cesàro average of infeasible-play probability eventually at least $\delta/2$ at every horizon. Every feasible reward is at most 1 and every infeasible reward is exactly -1, so each step's expected reward is at most $1-(1+M)\cdot\mathbb{P}(\text{infeasible at that step})$ for $M=1$; averaging this over a horizon T and comparing against optValue $= T$ gives regret at least $(1+M)(\delta/2)\cdot T$ eventually, which is linear with rate $(1+M)\delta/2 = \delta$. □

Remark 6.27 (Full proof)**.** *The quantitative version (**necessity_x5_explicit_bound**, Phase3.lean lines 1885–2045) and the clean corollary (**necessity_x5**, lines 2046–2069) are both available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (Appendix A). Hard-constraint bandit settings are an established area, e.g. Badanidiyuru et al. (2018); the specific minimal construction used here, with a fixed penalty and no budget mechanism, does not appear to be pre-stated there.*

Intuition — Public Policy

A regulatory or budget constraint is not a soft preference to be traded off against performance — it is a wall. An allocator who occasionally drifts into the forbidden range, even while doing well everywhere else, is not running a slightly-suboptimal version of a compliant strategy; it is running a strategy that periodically pays a fixed, large penalty for crossing a line that should never be crossed at all. A persistent habit of occasionally stepping over the line, rather than a one-off lapse, guarantees a bill that grows without bound the longer the allocator operates.

6.6 Ismail's Sixth Primitive — Feedback Adaptation (X_6)

Definition 6.28 (The regime-switching bandit)**.** *Fix a horizon T and gap $\Delta \in (0, \frac{1}{4}]$. The state is simply the elapsed time ($s_0 = 0$, $s \mapsto s+1$ deterministically). Let $\tau = \lfloor T/2 \rfloor$. Before τ, arm 1 pays $\frac{1}{2}+\Delta$ and arm 0 pays $\frac{1}{2}-\Delta$; after τ the two arms swap. A single, fixed changepoint flips which arm is better exactly once, halfway through the horizon.*

Lean 4 / Mathlib — Phase3.lean, lines 2078--2094

```
noncomputable def env₆_nonstat (T : ℕ) (Δ : ℝ) (hΔ0 : 0 < Δ) (hΔ4 : Δ≤1/4) :
    SixPrimitives.Env ℕ(Fin 2) Bool where
  trans := Kernel.deterministic (fun (s, _) => s + 1) (by measurability)
  obs := {
    toFun := fun (s, a) =>
      let postCP : Prop := T / 2 < s
      if (a = (0 : Fin 2)) = postCP
      then Phase2.bernoulliMeasure (1/2 + Δ) (by linarith) (by linarith)
      else Phase2.bernoulliMeasure (1/2 - Δ) (by linarith) (by linarith)
    measurable' := measurable_of_countable _
  }
  r := fun (s, a) =>
    let postCP : Prop := T / 2 < s
    if (a = (0 : Fin 2)) = postCP then 1/2 + Δelse 1/2 - Δ
  hr_meas := measurable_of_countable _
  μ₀:= Measure.dirac 0
  hμ₀ := inferInstance
```

Remark 6.29 (Two environments for two purposes)**.** *A second construction,* ***env$_6$_family*** *(Phase2.lean lines 3965–3991), packages the same changepoint as a literal* $\mathbb{N} \to \mathrm{Env}$ *sequence differing at* τ*, and is what verifies* HasP_6 *in Definition 3.1's literal sense (reproduced in Appendix B). The necessity theorem itself is proved against* ***env$_6$_nonstat*** *above, which encodes the identical structural failure — one changepoint, one swap — inside a single environment with time folded into the state, because that form is considerably more convenient for the trajectory-measure argument. Both constructions are the same structural failure told two ways for two different jobs.*

Definition 6.30 (The X_6 discriminator)**.** *Call arm* 1 stale *(it was optimal before* τ*, wrong after).* $\mathrm{staleProb}(t) := \mathbb{P}(A_t = 1)$ *under the law induced by running* alg *in* $\mathcal{E}_6(T, \Delta)$*.* alg *lacks* X_6 *if, for some* $\varepsilon \in (0, \frac{1}{2}]$ *and* $\delta > 0$ *with the post-change window* $(\tau, \lfloor(1+\varepsilon)\tau\rfloor]$ *nonempty,* $\mathrm{staleProb}(t) \geq \frac{1}{2} + \delta$ *throughout that window (Definition 2.10,* F_6*) — informally, the algorithm is still favoring the old answer, with better than even odds, well after it stopped being correct.*

Theorem 6.31 (Necessity of X_6)**.** *Fix* $\Delta \in (0, \frac{1}{4}]$*,* $\varepsilon \in (0, \frac{1}{2}]$*,* $\delta > 0$*, and* $\tau = \lfloor T/2 \rfloor$*. If* $\mathrm{staleProb}(t) \geq \frac{1}{2} + \delta$ *throughout* $(\tau, \lfloor(1+\varepsilon)\tau\rfloor]$*, then there is* $c > 0$ *and* $d \in \mathbb{R}$ *such that* $c \cdot T - d \leq \mathrm{regret}(\mathcal{E}_6(T, \Delta), \mathrm{alg}, T)$*, with* $c = \Delta(1 + 2\delta)(\varepsilon/2)$ *and* $d = \Delta(1 + 2\delta)(\varepsilon/2 + 1)$*.*

Lean 4 / Mathlib — `Phase3.lean, lines 2575--2589`

```
theorem necessity_x6
    {Sig : Type} [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass Sig] [MeasurableEq Sig]
    (T : ℕ)
    (Δ : ℝ) (hΔ0 : 0 < Δ) (hΔ4 : Δ≤1 / 4)
    (ε : ℝ) (hε0 : 0 < ε) (hε2 : ε≤1 / 2)
    (δ : ℝ) (hδ : 0 < δ)
    (tau : ℕ) (htau : tau = T / 2)
    (alg : SixPrimitives.Algorithm (Fin 2) Bool Sig)
    (hAlg : Phase2.AlgIsMarkov alg)
    (hLacks_post : ∀t : ℕ, tau < t →t ≤Nat.floor ((1 + ε) * (tau : ℝ)) →
      (Phase2.staleProbSeq (env₆_nonstat T ΔhΔ0 hΔ4) alg
        (env₆_nonstat T ΔhΔ0 hΔ4).hr_meas 1 t) ≥1 / 2 + δ) :
    ∃c > 0, ∃d : ℝ, c * (T : ℝ) - d ≤SixPrimitives.regret (env₆_nonstat T ΔhΔ0 hΔ4) alg T
```

Proof sketch. Throughout the post-change window, favoring the stale arm with probability at least $\frac{1}{2} + \delta$ costs at least $2\delta\Delta$ in expected reward per step relative to the now-correct arm (the gap between the two arms is 2Δ, and the excess probability mass on the wrong one is δ above even odds). The window has length on the order of $\varepsilon\tau = \varepsilon T/2$, so the accumulated cost over the window alone is already linear in T; the explicit-bound theorem (`necessity_x6_explicit_bound`, lines 2437–2574) carries this through exactly, and `necessity_x6` repackages the constants into the stated c, d form; both are available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (Appendix A). □

Remark 6.32 (Prior art)**.** *This is the single-changepoint specialization of the switching-bandit analysis of Garivier and Moulines (2011), an entire established subfield since 2011.*

Intuition — Investing

A regime change in markets — a rate environment shift, a structural break in a relationship the strategy depended on — doesn't announce itself with a label. A strategy that keeps allocating to what worked in the old regime, with real conviction, well into the new one isn't being patient; it is paying the gap between the old winner and the new winner, every period, for as long as the stale belief persists. The fix isn't forecasting the regime change in advance — it's not clinging to old evidence past the point where it stopped describing the world, and that is a property of the decision process, not of how good

its predictions were going in.

This completes Part I. All six primitives are necessary, each witnessed by its own minimal environment, each using only the weak (ordinary) sense of lacking the primitive — the strongest form of the claim available.

7 Part II — Ismail's Proof of Primitive Independence

For each ordered pair (i, j) with $i \neq j$, the algorithm built to handle X_j's own structural failure is shown to lack X_i when redeployed in X_i's home environment — the same six environments from Part I, with no new construction needed. Thirty such pairs cover every off-diagonal cell of a 6×6 grid.

7.1 The Challengers

Every challenger is built from one template: a fixed (possibly randomized) policy that never updates its summary at all.

Lean 4 / Mathlib — `Phase4.lean, lines 28--62`

```
noncomputable def canonical_challenger (A O : Type)
    [MeasurableSpace A] [MeasurableSpace O]
    (policy : Kernel Unit A) : SixPrimitives.Algorithm A O Unit where
  act := policy
  update := Kernel.const _ (Measure.dirac ())
  σ₀ := ()

-- The six are this same template under six different names, each
-- instantiated with whichever fixed policy handles its own primitive:
noncomputable def alg1_bayes (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
noncomputable def alg2_safe (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
noncomputable def alg3_bridge (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
noncomputable def alg4_ucb (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
noncomputable def alg5_feas (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
noncomputable def alg6_shortMem (A O : Type) [...] (policy : Kernel Unit A) :=
    canonical_challenger A O policy
```

Lean 4 / Mathlib — `Phase4.lean, lines 66--86`

```
noncomputable abbrev pol0 : Kernel Unit (Fin 2) := Kernel.const _ (Measure.dirac 0)
noncomputable abbrev pol1 : Kernel Unit (Fin 2) := Kernel.const _ (Measure.dirac 1)
noncomputable abbrev polR : Kernel Unit ℝ := Kernel.const _ (Measure.dirac (1/2 : ℝ))

noncomputable def pol_eps : Kernel Unit (Fin 2) :=
  Kernel.const _ ((1/2 : ℝ≥0∞) •Measure.dirac 0 + (1/2 : ℝ≥0∞) •Measure.dirac 1)
```

Remark 7.1 (Why independence falls out almost for free)**.** *Each primitive's own structural failure, per Part I, is survivable by some* fixed*, non-adaptive policy: always play it safe in the trap (*pol0 *in* $\mathcal{E}_2$*), never*

*gamble on the bridge (*pol0 *in $\mathcal{E}_3$), never play the forbidden midpoint (*polR *avoiding $\frac{1}{2}$ is, ironically, the constant action exactly* at $\frac{1}{2}$ *used here to fail X_5 maximally elsewhere — see the table). A policy that is fixed in this sense carries no information about which environment it is actually in, so it is exactly the kind of object Part I's necessity proofs already rule out for every* other *structural failure: a constant action gives zero total-variation distance between two environments (X_1), a constant conditional probability of any fixed action (X_2, X_4, X_6), zero cumulative attempts at any fixed action (X_3), or certainty of whichever action happens to be fixed (X_5). Independence here is a direct consequence of necessity, not a separate phenomenon.*

7.2 Two Worked Examples

Example 7.2 (A deterministic challenger fails X_1)**.** alg2_safe*, redeployed on X_1's home turf with the constant policy* pol0*, always plays arm* 0 *regardless of which of $\mathcal{E}_1^{(0)}, \mathcal{E}_1^{(1)}$ it is actually in. The two action marginals at every horizon are therefore identical point masses, the total variation distance between them is identically* 0*, and* LacksX_1 *holds with* $B = 0$.

Lean 4 / Mathlib — Phase4.lean, lines 111--162

```
lemma alg2_lacks_x1 (bp : Phase2.BanditParam) :
    LacksX₁ (Phase3.miSeq (Phase2.env₁_0 bp) (Phase2.env₁_1 bp)
      (alg2_safe (Fin 2) Bool pol0) (Phase2.env₁_0_isMarkov bp)
      (Phase2.env₁_1_isMarkov bp) (alg2_isMarkov (Fin 2) Bool pol0)) := by
  unfold LacksX₁
  refine ⟨0, by norm_num, ?_⟩
  -- the action marginal under a constant policy is the same point mass
  -- in both environments, at every horizon, so miSeq ≡0 identically
  ...
  simp [Filter.limsup_const]
```

Example 7.3 (A randomized challenger fails X_2)**.** *Any challenger built from* pol_eps *(the fixed* 50/50 *randomizer) plays the dangerous action with conditional probability exactly $\frac{1}{2}$ at every step, regardless of survival history — since the policy never looks at its own summary at all. This is proved once, generically, and reused for all five algorithms that are tested against X_2's home turf using* pol_eps.

Lean 4 / Mathlib — Phase4.lean, lines 746--769

```
lemma canonical_challenger_lacks_x2 :
    LacksX₂ (Phase2.condDangerProbSeq K hK
      (canonical_challenger (Fin 2) (Fin (K+2)) pol_eps)
      (canonical_challenger_isMarkov (Fin 2) (Fin (K+2)) pol_eps)
      (env₂_dangerAction K)) := by
  unfold LacksX₂
  use 1 / 2
  constructor
  ·norm_num
  ·intro k
    unfold Phase2.condDangerProbSeq
    -- the joint (danger ∧survival) probability is exactly half the
    -- survival probability at every step, since the policy is pol_eps
    -- independent of everything ⟹ conditional danger prob ≡1/2
    ...

lemma alg1_lacks_x2 : LacksX₂ (Phase2.condDangerProbSeq K hK
    (alg1_bayes (Fin 2) (Fin (K+2)) pol_eps)
    (alg1_isMarkov (Fin 2) (Fin (K+2)) pol_eps)
```

```
    (env₂_dangerAction K)) := canonical_challenger_lacks_x2 K hK
-- alg3, alg4, alg5, alg6 follow by the identical one-line citation
```

7.3 All Thirty Pairs

Home turf	Lacked by	Witnessing policy	Phase4.lean lines
$\mathcal{E}_1$ (vs. X_1)	algs 2,3,4,5,6	pol0 (constant arm 0)	103–407
$\mathcal{E}_2$ (vs. X_2)	algs 1,3,4,5,6	pol_eps (constant 50/50)	663–769
$\mathcal{E}_3$ (vs. X_3)	algs 1,2,4,5,6	pol0 (never bridges)	773–931
$\mathcal{E}_4$ (vs. X_4)	algs 1,2,3,5,6	pol_eps (never commits)	1083–1137
$\mathcal{E}_5$ (vs. X_5)	algs 1,2,3,4,6	polR (constant action $\frac{1}{2}$, always forbidden)	1149–1399
$\mathcal{E}_6$ (vs. X_6)	algs 1,2,3,4,5	pol0 (never adapts)	1403–1475

Table 3: The thirty home-turf failures. pol_eps on $\mathcal{E}_2$ and $\mathcal{E}_4$, and the constant-zero policy on $\mathcal{E}_6$, are each proved once generically (`canonical_challenger_lacks_x2`, `canonical_challenger_lacks_x4`, `deterministic_zero_lacks_x6`) and reused across all five algorithms in that row by direct citation.

Theorem 7.4 (Mutual independence)**.** *All thirty pairs hold simultaneously.*

Lean 4 / Mathlib — `Phase4.lean, lines 1480--1532`

```
theorem mutual_independence
    (bp : Phase2.BanditParam) (K : ℕ) (hK : 0 < K)
    (p : ℝ) (hp0 : 0 ≤p) (hp1 : p ≤1)
    (T₀ : ℕ) (Δ : ℝ) (hΔ0 : 0 < Δ) (hΔ4 : Δ≤1/4) :
    -- X1 lacked by Algs 2,3,4,5,6
    LacksX₁ (... alg2_safe ...) ∧LacksX₁ (... alg3_bridge ...) ∧
    LacksX₁ (... alg4_ucb ...) ∧LacksX₁ (... alg5_feas ...) ∧
    LacksX₁ (... alg6_shortMem ...) ∧
    -- X2 lacked by Algs 1,3,4,5,6
    LacksX₂ (... alg1_bayes ...) ∧LacksX₂ (... alg3_bridge ...) ∧
    LacksX₂ (... alg4_ucb ...) ∧LacksX₂ (... alg5_feas ...) ∧
    LacksX₂ (... alg6_shortMem ...) ∧
    -- X3 lacked by Algs 1,2,4,5,6
    LacksX₃ (... alg1_bayes ...) ∧LacksX₃ (... alg2_safe ...) ∧
    LacksX₃ (... alg4_ucb ...) ∧LacksX₃ (... alg5_feas ...) ∧
    LacksX₃ (... alg6_shortMem ...) ∧
    -- X4 lacked by Algs 1,2,3,5,6
    LacksX₄ (... alg1_bayes ...) ∧LacksX₄ (... alg2_safe ...) ∧
    LacksX₄ (... alg3_bridge ...) ∧LacksX₄ (... alg5_feas ...) ∧
    LacksX₄ (... alg6_shortMem ...) ∧
    -- X5 lacked by Algs 1,2,3,4,6
    LacksX₅ (... alg1_bayes ...) ∧LacksX₅ (... alg2_safe ...) ∧
    LacksX₅ (... alg3_bridge ...) ∧LacksX₅ (... alg4_ucb ...) ∧
    LacksX₅ (... alg6_shortMem ...) ∧
    -- X6 lacked by Algs 1,2,3,4,5
    LacksX₆ (... alg1_bayes ...) ∧LacksX₆ (... alg2_safe ...) ∧
    LacksX₆ (... alg3_bridge ...) ∧LacksX₆ (... alg4_ucb ...) ∧
    LacksX₆ (... alg5_feas ...) := by
```

```
refine ⟨alg2_lacks_x1 bp, alg3_lacks_x1 bp, alg4_lacks_x1 bp, alg5_lacks_x1 bp,
    alg6_lacks_x1 bp,
  alg1_lacks_x2 K hK, alg3_lacks_x2 K hK, alg4_lacks_x2 K hK, alg5_lacks_x2 K hK,
      alg6_lacks_x2 K hK,
  alg1_lacks_x3 p hp0 hp1, alg2_lacks_x3 p hp0 hp1, alg4_lacks_x3 p hp0 hp1,
  alg5_lacks_x3 p hp0 hp1, alg6_lacks_x3 p hp0 hp1,
  alg1_lacks_x4, alg2_lacks_x4, alg3_lacks_x4, alg5_lacks_x4, alg6_lacks_x4,
  alg1_lacks_x5, alg2_lacks_x5, alg3_lacks_x5, alg4_lacks_x5, alg6_lacks_x5,
  alg1_lacks_x6 T₀ ΔhΔ0 hΔ4, alg2_lacks_x6 T₀ ΔhΔ0 hΔ4, alg3_lacks_x6 T₀ ΔhΔ0 hΔ4,
  alg4_lacks_x6 T₀ ΔhΔ0 hΔ4, alg5_lacks_x6 T₀ ΔhΔ0 hΔ4⟩
```

Remark 7.5 (Faithfulness of the abbreviation)**.** *The ⋯ elisions above stand for the full applied arguments shown verbatim in Section 7.2 and Table 3 (environment, algorithm, and Markov witnesses); nothing is elided in the actual repository. The unabridged statement is available at https://github.com/M-Ismail-ZA/IsmailsPrimitives (Appendix A lists every one of the thirty component lemmas individually).*

Intuition — Operations

A fraud-detection model is not, by virtue of being good at fraud detection, also a model for regulatory-capital allocation, even though both get filed under "risk management." Competence at one specific structural failure is not a general-purpose property that happens to transfer; it is the result of a specific adaptation to a specific problem, and the simplest version of that adaptation — the fixed rule that handles this one thing and nothing else — is definitionally blind to every other problem in the building. None of the six primitives is a proxy for any of the others; each has to be separately earned.

8 Part III — Ismail's Formally-Verified Bridge Towards Sufficiency

8.1 What This Part Proves, and What It Defers

This structure follows from an asymmetry between what Necessity and Sequential Dependence each claim. Failure has one shape: an algorithm either has the structural property survival requires, or it does not — and if not, the outcome is linear regret, full stop, independent of what else the algorithm does well or how success is defined. This is why Part I is unconditional throughout: necessity closes the moment the structural failure is granted, requiring nothing about the algorithm's aims beyond surviving it. Success has no comparable single shape; what sufficiency looks like depends on the environment, the goal variable, and choices no necessity argument can make on an implementer's behalf. Part III is honestly conditional for the same reason a general theory of success could never be otherwise: it proves the chain's logical architecture is sound and states exactly which further choices — a concrete algorithm, environment, and goal variable — would close each remaining link, rather than asserting one universal closing move exists.

The methodological preface below is stated once and applies to all six links. Each link $X_i \to X_{i+1}$ is established by three results:

- a **forward** theorem: under stated structural hypotheses, the mutual information between the canonical summary and a goal variable advances toward its maximum;
- a **reverse** theorem: under a stated, explicitly named bridging hypothesis connecting failure of the primitive to information erosion, the information ceiling is strictly below the maximum;
- a **non-reversibility** result: a fully concrete, no-hypotheses-deferred construction showing that satisfying the next primitive's metric without the current one is achievable but informationally hollow.

Every named hypothesis that is not discharged within this paper is flagged in an Implementation Obligation box at the point it is used, stating exactly what a concrete algorithm and environment would need to show to invoke the result — the constructive question left open for instantiation, not asserted to hold in general.

8.2 IET 1: $X_1 \to X_2$

Theorem 8.1 (IET 1, forward). *If* alg *robustly possesses* X_1 *(Definition 8.4's counterpart for* X_1*:* $\liminf_t I(G_t;\Sigma_t) > \frac{1}{2}$*) and its summary is sufficient for* G_t*, then* $\liminf_t I(G_t;\Sigma_t) > \frac{1}{2}$.

Lean 4 / Mathlib — Phase5.lean, lines 222--229

```
theorem iet1_forward
    (alg : SixPrimitives.Algorithm A O Sig)
    [Phase2.AlgIsDeterministic alg]
    (hAlg : Phase2.AlgIsMarkov alg)
    (_hRobustX1 : RobustX1 env alg hEnv hAlg goal_var hGoalMeas)
    (_hS4 : IsSufficientSummary env alg hEnv hAlg goal_var hGoalMeas) :
    1 / 2 < liminf (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop := by
  exact _hRobustX1
```

No Implementation Obligation beyond Robust X_1 itself

Unlike IET 2's forward link, this one needs no separate bridging step: RobustX$_1$ is *defined* as exactly the conclusion ($\liminf_t I(G_t;\Sigma_t) > \frac{1}{2}$), so the theorem is a restatement, not a derivation. The sufficiency hypothesis is carried for uniformity with the rest of the chain but is not needed here. All of the substantive content is in establishing robust X_1 itself for a concrete algorithm — which is exactly what Part I's necessity result for X_1 characterizes the failure mode of.

Theorem 8.2 (IET 1, reverse). *If* alg *lacks* X_1*, then* $\exists B < \frac{1}{2}$ *with* $\limsup_t I(G_t;\Sigma_t) \le B$.

Lean 4 / Mathlib — Phase5.lean, lines 231--237

```
theorem iet1_reverse
    (alg : SixPrimitives.Algorithm A O Sig)
    [Phase2.AlgIsDeterministic alg]
    (hAlg : Phase2.AlgIsMarkov alg)
    (_hLacksX1 : SixPrimitives.LacksX₁ (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas))
        :
    ∃B < 1/2, limsup (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop ≤B := by
  exact _hLacksX1
```

Theorem 8.3 (IET 1, non-reversibility). *There is an environment, a (constant) goal variable, and an algorithm with* $I(G_t;\Sigma_t) = 0 \le \frac{1}{4}$ *for every* t.

Lean 4 / Mathlib — Phase5.lean, lines 239--264

```
theorem iet1_non_reversible :
    ∃(env_12 : SixPrimitives.Env Unit (Fin 2) Bool)
      (hEnv_12 : Phase2.EnvIsMarkov env_12)
      (g_12 : ∀T, Phase2.Trajectory (Fin 2) Bool T →Fin 2)
      (hG_12 : ∀T, Measurable (g_12 T)),
    ∃B < 1/2, ∀t, MI_Summary_Seq env_12
        (Phase4.alg2_safe (Fin 2) Bool Phase4.pol0) hEnv_12
        (Phase4.alg2_isMarkov (Fin 2) Bool Phase4.pol0) g_12 hG_12 t ≤B := by
```

```
  have bp : Phase2.BanditParam := ⟨1/4, by norm_num, by norm_num⟩
  refine ⟨Phase2.env₁_0 bp, Phase2.env₁_0_isMarkov bp, fun _ _ => 0,
    fun _ => measurable_const, 1/4, by norm_num, fun t => ?_⟩
  -- g_12 is the constant goal variable "0": zero entropy regardless of
  -- the summary, so mutual information is identically 0
  ...
```

Intuition — Investing

This link is the cleanest of the six precisely because robust X_1 *is* the information target, not a precondition for reaching it: a desk that robustly tracks which of two strategies is true is, by definition, carrying more than half a bit of information about the answer. There's no separate translation step from "avoids the known trap" to "has the information"—for X_1 those are the same statement. The non-reversibility result is the same caution as elsewhere: a "signal" that's actually a constant tells you nothing, no matter how it's measured.

8.3 IET 2: $X_2 \to X_3$

This link is worked through in full below, as the template the remaining five links (and the closing, martingale-based link $X_6 \to X_1$) will follow.

8.3.1 Recalled definitions

Definition 8.4 (Robust possession of X_2)**.** *Fix a checkpoint sequence $\tau : \mathbb{N} \to \mathbb{N}$ and a designated dangerous action a_{danger}. An algorithm* robustly possesses X_2 *relative to $(\tau, a_{\text{danger}})$ if the sequence of probabilities it assigns to a_{danger}, evaluated at the checkpoints $\tau(k)$, is summable:*

$$\sum_{k=0}^{\infty} \mathbb{P}\big(A_{\tau(k)} = a_{\text{danger}}\big) < \infty.$$

Intuition — Operations

Summability is a stronger demand than "the danger probability goes to zero." It says the algorithm's flirtation with the catastrophic action is not just shrinking but shrinking fast enough that, by Borel–Cantelli, it touches that action only finitely many times with probability one. A trading desk that merely reduces its exposure to a blow-up trade over time is not the same as a desk that, with certainty in the long run, stops taking that trade altogether.

Lean 4 / Mathlib — `Phase5.lean, lines 114--124`

```
def RobustX2 {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [MeasurableSingletonClass A]
    (env : SixPrimitives.Env S A O)
    (alg : SixPrimitives.Algorithm A O Sig)
    (hEnv : Phase2.EnvIsMarkov env)
    (hAlg : Phase2.AlgIsMarkov alg)
    (tau : ℕ→ℕ)
    (a_danger : A) : Prop :=
  RobustX2_general env alg hEnv hAlg tau a_danger
```

Definition 8.5 (Robust possession of X_3)**.** *Fix a designated bridge action a_B. An algorithm* robustly possesses X_3 *relative to a_B if, for every threshold $M > 0$, the probability that its cumulative count of bridge-action plays up to horizon T exceeds $M\sqrt{T}$ tends to one as $T \to \infty$:*

$$\forall M > 0, \quad \mathbb{P}\Big(\textstyle\sum_{t<T} \mathbf{1}[A_t = a_B] \;\geq\; M\sqrt{T}\Big) \xrightarrow[T\to\infty]{} 1.$$

Lean 4 / Mathlib — `Phase5.lean, lines 126--150`

```
noncomputable def bridge_count_rv {A O : Type*} [DecidableEq A]
    (a_B : A) (T : ℕ) (ω : Phase2.Trajectory A O T) : ℝ:=
  ∑t : Fin T, if (ω t).1 = a_B then 1 else 0

def RobustX3_ae {S A O Sig : Type*}
    [MeasurableSpace S] [MeasurableSpace A] [MeasurableSpace O]
    [MeasurableSpace Sig] [TopologicalSpace Sig] [BorelSpace Sig]
    [DecidableEq A]
    (env : SixPrimitives.Env S A O)
    (alg : SixPrimitives.Algorithm A O Sig)
    (a_B : A) : Prop :=
  ∀M > 0, Tendsto (fun T : ℕ=>
    (Phase2.trajMeasure env alg env.hr_meas T
      {ω | bridge_count_rv a_B T ω≥M * Real.sqrt T}).toReal
  ) atTop (nhds 1)
```

8.3.2 Forward: robust X_2, X_3 carry full information

Theorem 8.6 (IET 2, forward)**.** *Let G_t be a binary goal variable and suppose the algorithm's canonical summary is a sufficient statistic for G_t at every horizon t (i.e. conditioning on the summary loses no information about G_t relative to conditioning on the full history), and that G_t retains its full* $\log 2$ *nats of entropy at every horizon. Then the mutual information between the summary and G_t is identically* $\log 2$ *from the start, and in particular*

$$I(G_t; \Sigma_t) \xrightarrow[t\to\infty]{} \log 2.$$

Proof sketch. Sufficiency gives $H(G_t \mid \Sigma_t) = H(G_t \mid \text{full history})$ at every t; since the algorithm is a deterministic function of its own summary and history, the right-hand side is 0. So $I(G_t; \Sigma_t) = H(G_t) - H(G_t \mid \Sigma_t) = H(G_t) - 0 = \log 2$ for every t, hence the sequence is the constant sequence $\log 2$, which trivially converges to $\log 2$. □

Implementation Obligation

The theorem is stated for algorithms that robustly possess X_2 and X_3 in the relevant environment — that is its intended domain of application — but the two hypotheses actually doing the logical work are *sufficiency of the summary* and *entropy retention of G_t*. Nothing in this paper shows that robust X_2 and robust X_3 *imply* those two conditions in a general Class $\mathcal{C}$ environment. To instantiate this link for a concrete algorithm and environment, one must exhibit that implication directly: show that an algorithm robustly avoiding the dangerous action (robust X_2) and robustly exploring the bridge at rate $\omega(\sqrt{T})$ (robust X_3) is exactly what makes its canonical summary sufficient for G_t and keeps G_t's entropy at $\log 2$ in that environment. That construction is environment-specific and is left to the implementer.

Lean 4 / Mathlib — Phase5.lean, lines 278--307

```
theorem iet2_forward
    (alg : SixPrimitives.Algorithm A O Sig)
    [Phase2.AlgIsDeterministic alg]
    (hAlg : Phase2.AlgIsMarkov alg)
    (_hRobustX2 : RobustX2 env alg hEnv hAlg tau a_danger)
    (_hRobustX3 : RobustX3_ae env alg a_B)
    (_hS4 : IsSufficientSummary env alg hEnv hAlg goal_var hGoalMeas)
    (hGoalEnt : ∀t : ℕ,
      let μ:= Phase2.trajMeasure env alg env.hr_meas t
      letI : MeasureSpace (Phase2.Trajectory A O t) := ⟨μ⟩
      letI : IsProbabilityMeasure μ:=
        Phase2.trajMeasure_isProbability env alg env.hr_meas hEnv hAlg t
      Phase2.entropy (goal_var t) = Real.log 2) :
    Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas)
      atTop (nhds (Real.log 2)) := by
  have h_seq_eq : MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas =
      fun _ => Real.log 2 := by
    ext t
    let μ:= Phase2.trajMeasure env alg env.hr_meas t
    letI : MeasureSpace (Phase2.Trajectory A O t) := ⟨μ⟩
    letI hProb : IsProbabilityMeasure μ:=
      Phase2.trajMeasure_isProbability env alg env.hr_meas hEnv hAlg t
    unfold MI_Summary_Seq
    dsimp only
    rw [_hS4 t]
    have h_det : Phase2.condEntropy (goal_var t) ⊤= 0 := by
      have h_eq : goal_var t = id ∘(goal_var t) := rfl
      rw [h_eq]
      apply Phase2.condEntropy_of_deterministic id measurable_id
        (goal_var t) (hGoalMeas t) ⊤
      exact le_top
    rw [h_det, sub_zero]
    exact hGoalEnt t
  rw [h_seq_eq]
  exact tendsto_const_nhds
```

8.3.3 Reverse: failing X_2 caps the information ceiling

Theorem 8.7 (IET 2, reverse)**.** *Suppose the algorithm does* not *robustly possess X_2, that G_t retains its full $\log 2$ nats of entropy at every horizon, and that failing to robustly avoid the dangerous action eventually forces the conditional entropy of G_t given the summary to remain at least $\delta \log 2$, for some $\delta > 0$. Then*

$$\limsup_{t\to\infty} I(G_t;\Sigma_t) \;\le\; (1-\delta)\log 2,$$

strictly below the maximum.

Proof sketch. The third hypothesis gives, eventually in t, $H(G_t \mid \Sigma_t) \ge \delta \log 2$; combined with $H(G_t) = \log 2$ this gives $I(G_t;\Sigma_t) \le (1-\delta)\log 2$ eventually. Mutual information is also bounded below (by data processing, $H(G_t \mid \Sigma_t) \le H(G_t \mid \top)$, giving a cobound), so the eventual bound passes to the lim sup. □

Implementation Obligation

The genuinely deferred content here is the bridging hypothesis itself: that failing to robustly avoid the dangerous action *forces* conditional entropy to stay above a fixed fraction of $\log 2$. This is an environment-specific causal claim — it says the act of repeatedly risking the dangerous action is itself what destroys the algorithm's ability to learn G_t — and constructing a concrete environment and algorithm pair in which this implication holds is left to the implementer. The theorem guarantees that, once that mechanism is established, the quantitative information ceiling follows rigorously.

8.3.4 Non-reversibility: a hollow victory, fully closed

Theorem 8.8 (IET 2, non-reversibility)**.** *There exists an environment, a (constant) goal variable, and an algorithm such that the algorithm satisfies X_3's defining metric by construction, yet*

$$I(G_t; \Sigma_t) = 0 \;\leq\; \tfrac{1}{2}\log 2 \qquad \textit{for every } t.$$

Proof sketch. Take G_t to be the constant goal variable "always true." A constant random variable has zero entropy, so both $H(G_t)$ and $H(G_t \mid \Sigma_t)$ vanish for every t and every summary, regardless of what the algorithm does. Mutual information is therefore identically zero, which is certainly $\leq \frac{1}{2}\log 2$. □

No Implementation Obligation

Unlike the previous two results, this construction is fully concrete: the environment, the goal variable, and the algorithm are all named explicitly, and the bound is verified outright with no deferred hypothesis. It demonstrates only that a degenerate choice of goal variable trivializes the entire chain — which is precisely the point: any real instantiation of this paper's pipeline must choose a goal variable that actually carries information, or "satisfying X_3" is vacuous.

Lean 4 / Mathlib — `Phase5.lean, lines 362--391`

```
theorem iet2_non_reversible (K : ℕ) (_hK : 0 < K) :
    ∃(env_23 : SixPrimitives.Env (Fin (K+2)) (Fin 2) (Fin (K+2)))
      (hEnv_23 : Phase2.EnvIsMarkov env_23)
      (g_23 : ∀T, Phase2.Trajectory (Fin 2) (Fin (K+2)) T →Bool)
      (hG_23 : ∀T, Measurable (g_23 T)),
    ∃δ> 0, ∀t, MI_Summary_Seq env_23
        (Phase4.alg3_bridge (Fin 2) (Fin (K+2)) Phase4.pol0)
        hEnv_23 (Phase4.alg3_isMarkov (Fin 2) (Fin (K+2)) Phase4.pol0)
        g_23 hG_23 t ≤(1 - δ) * Real.log 2 := by
  refine ⟨Phase2.env₂ K _hK,
    Phase2.envIsDeterministic_isMarkov _ (Phase2.env₂_isDet K _hK),
    fun _ _ => true, fun _ => measurable_const, 1/2, by norm_num, fun t => ?_⟩
  let μ:= Phase2.trajMeasure (Phase2.env₂ K _hK)
    (Phase4.alg3_bridge (Fin 2) (Fin (K+2)) Phase4.pol0) (Phase2.env₂ K _hK).hr_meas t
  letI : MeasureSpace (Phase2.Trajectory (Fin 2) (Fin (K+2)) t) := ⟨μ⟩
  letI : IsProbabilityMeasure μ:= Phase2.trajMeasure_isProbability (Phase2.env₂ K _hK)
    (Phase4.alg3_bridge (Fin 2) (Fin (K+2)) Phase4.pol0) (Phase2.env₂ K _hK).hr_meas
    (Phase2.envIsDeterministic_isMarkov _ (Phase2.env₂_isDet K _hK))
    (Phase4.alg3_isMarkov (Fin 2) (Fin (K+2)) Phase4.pol0) t
  -- the constant goal variable factors through Unit, which has zero
  -- entropy unconditionally and zero entropy conditional on anything
  have h_eq : (fun _ : Phase2.Trajectory (Fin 2) (Fin (K+2)) t => true) =
              (fun _ : Unit => true) ∘(fun _ => ()) := rfl
  have h_comap_bot :
      MeasurableSpace.comap (fun _ : Phase2.Trajectory (Fin 2) (Fin (K+2)) t => ())
          inferInstance ≤
```

```
      (⊥ : MeasurableSpace (Phase2.Trajectory (Fin 2) (Fin (K+2)) t)) :=
  Measurable.comap_le
    (@measurable_const Unit (Phase2.Trajectory (Fin 2) (Fin (K+2)) t) inferInstance ⊥())
have h_ent_zero : Phase2.entropy (fun _ : Phase2.Trajectory (Fin 2) (Fin (K+2)) t => true)
    = 0 := by
  rw [Phase2.entropy, h_eq]
  exact Phase2.condEntropy_of_deterministic (fun _ : Unit => true) measurable_const
    (fun _ => ()) measurable_const ⊥h_comap_bot
have h_cond_ent_zero : Phase2.condEntropy
    (fun _ : Phase2.Trajectory (Fin 2) (Fin (K+2)) t => true)
    (MeasurableSpace.comap
      (Phase2.summary_at (Phase4.alg3_bridge (Fin 2) (Fin (K+2)) Phase4.pol0) t t (le_refl t
          ))
      inferInstance) = 0 := by
  rw [h_eq]
  exact Phase2.condEntropy_of_deterministic (fun _ : Unit => true) measurable_const
    (fun _ => ()) measurable_const _ (h_comap_bot.trans bot_le)
unfold MI_Summary_Seq
dsimp only
rw [h_ent_zero, h_cond_ent_zero]
have h_log_pos : 0 ≤Real.log 2 := Real.log_nonneg (by norm_num)
linarith
```

8.3.5 Cross-domain reading

Intuition — Consulting

A turnaround consultant who has not yet gotten a client to stop touching the move that keeps causing write-downs (no robust X_2) cannot be credited with having found the client's path to a better market segment, even while running pilots in that segment (the X_3 behaviour). Until the dangerous move stops, any signal from the pilots is contaminated by the ongoing damage and tells you little about which segment is actually better. Once it stops, the pilots' results really do start to mean something — that is the forward link. And if the client is using a "pilot" that was never designed to teach anyone anything (the constant-goal-variable construction), running it proves nothing about either primitive, no matter how it's dressed up in a report.

8.4 IET 3: $X_3 \to X_4$

Theorem 8.9 (IET 3, forward)**.** *If* alg *robustly possesses* X_3*, its summary is sufficient for* G_t*, and robust* X_3 *entails entropy convergence (*$H(G_t) \to \log 2$*), then* $I(G_t; \Sigma_t) \to \log 2$*.*

Lean 4 / Mathlib — `Phase5.lean, lines 404--416`

```
theorem iet3_forward
    (alg : SixPrimitives.Algorithm A O Sig)
    [Phase2.AlgIsDeterministic alg]
    (hAlg : Phase2.AlgIsMarkov alg)
    (_hRobustX3 : RobustX3_ae env alg a_B)
    (_hS4 : IsSufficientSummary env alg hEnv hAlg goal_var hGoalMeas)
    (h_convergence : RobustX3_ae env alg a_B →
      Tendsto (fun t => ... Phase2.entropy (goal_var t)) atTop (nhds (Real.log 2))) :
    Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds (Real.log 2))
```

Implementation Obligation

Note the shift from IET 2's forward link: there, the deferred condition was a pointwise equality ($H(G_t) = \log 2$ at every t); here it is an asymptotic statement ($H(G_t) \to \log 2$). The proof still reduces $I(G_t; \Sigma_t)$ to $H(G_t)$ exactly via sufficiency (the conditional term vanishes identically, as in every forward link so far), so *convergence* of the mutual information is exactly as strong as *convergence* of the entropy term — which is what `h_convergence` must establish for a concrete instantiation.

The reverse link follows the same shape as IET 2's (a deferred `h_bridge_fails` hypothesis converts $\neg\text{RobustX}_3$ into a quantitative conditional-entropy floor, then a real lim sup calculation closes the bound), and the non-reversibility result is again fully concrete — a one-state, zero-reward environment with a constant goal variable, giving $I(G_t; \Sigma_t) \to 0$ outright.

Lean 4 / Mathlib — Phase5.lean, lines 489--507 (abridged)

```
theorem iet3_non_reversible :
  ∃(env_34 : SixPrimitives.Env Unit (Fin 2) Unit) (hEnv_34 : ...)
    (g_34 : ∀T, Phase2.Trajectory (Fin 2) Unit T →Fin 2) (hG_34 : ...),
  Tendsto (MI_Summary_Seq env_34 (Phase4.alg4_ucb (Fin 2) Unit Phase4.pol0)
    hEnv_34 ... g_34 hG_34) atTop (nhds 0) := by
  let env_34 : SixPrimitives.Env Unit (Fin 2) Unit :=
    { trans := Kernel.const _ (Measure.dirac ()), obs := Kernel.const _ (Measure.dirac ()),
      r := fun _ => 0, hr_meas := measurable_const,
      μ₀:= Measure.dirac (), hμ₀ := inferInstance }
  let g_34 : ∀T, Phase2.Trajectory (Fin 2) Unit T →Fin 2 := fun T _ => 0
  ...
```

8.5 IET 4: $X_4 \to X_5$, and IET 5: $X_5 \to X_6$

These two links are stated at a deliberately higher level of abstraction than the other four. Where IET 1–3 and IET 6 name a concrete possession predicate (RobustX_1, RobustX_3_ae, RobustX_6_ae) and, in their non-reversibility results, a concrete environment, IET 4 and IET 5 leave *both* fully abstract: the possession hypothesis is an arbitrary proposition P_{RobustX_4} (resp. P_{RobustX_5}), and even the non-reversibility result is a logical relay rather than a witnessed construction.

Lean 4 / Mathlib — Phase5.lean, lines 553--612 (IET 4, all three results)

```
theorem iet4_forward
    {P_RobustX4 P_Sufficient : Prop}
    (hRobustX4 : P_RobustX4) (hSufficient : P_Sufficient)
    (h_pipeline_advances : P_RobustX4 →P_Sufficient →
      Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds (Real.log 2)))
          :
    Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds (Real.log 2))
        := by
  exact h_pipeline_advances hRobustX4 hSufficient

theorem iet4_reverse
    {P_RobustX4 : Prop} (hLacksX4 : ¬P_RobustX4)
    (h_danger_erodes : ¬P_RobustX4 →
      Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds 0)) :
    Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds 0) := by
  exact h_danger_erodes hLacksX4
```

```
theorem iet4_non_reversible
    {P_RobustX5 : Prop} (h_isolated_X5 : P_RobustX5)
    (h_degenerate : ∀T traj, goal_var T traj = g_const)
    (h_meaningless_progress : P_RobustX5 →(∀ T traj, goal_var T traj = g_const) →
      Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds 0)) :
    Tendsto (MI_Summary_Seq env alg hEnv hAlg goal_var hGoalMeas) atTop (nhds 0) := by
  exact h_meaningless_progress h_isolated_X5 h_degenerate
```

A larger obligation than elsewhere

IET 5 (Phase5.lean, lines 624–682) repeats this same fully-abstract pattern verbatim, with P_{RobustX_5} in place of P_{RobustX_4}. For these two links, instantiating the chain concretely requires strictly more work than for the others: an implementer must supply not only the bridging implication (as everywhere else) but also the concrete definition of what robust possession even *means* at this link, since none is fixed in the theorem itself. This is a genuine difference in how much these two links commit to, not a stylistic accident, and this paper states it as such rather than presenting all six links as uniformly specified.

Intuition — Public Policy

Some links in a causal chain are well-characterized enough to name precisely — "stops touching the dangerous lever" is a concrete, checkable behavior. Others are real but resist a single canonical formalization across every possible setting — what "robustly respecting a constraint" or "robustly committing to a decision" means can reasonably differ by domain. Rather than force a one-size formal definition onto $X_4 \to X_5$ and $X_5 \to X_6$, the chain is kept honest by leaving the shape of the requirement explicit and the content open, to be filled in by whoever is closest to the concrete setting.

8.6 IET 6: $X_6 \to X_1$ — The Accumulative Closure

The sixth link closes the cycle back to X_1 and is the only one that makes the "compounds across cycles" promise precise, via an actual Doob martingale construction over a filtration indexed by cycles of length $L_{\min}$.

Lean 4 / Mathlib — Phase5.lean, lines 707--737

```
noncomputable def cycle_filtration (K_total L_min : ℕ) :
    MeasureTheory.Filtration ℕ(inferInstance : MeasurableSpace (Phase2.Trajectory A O (
        K_total * L_min))) where
  seq k := ⨆(i : Fin (K_total * L_min)) (_hi : i.val < k * L_min),
    MeasurableSpace.comap (fun ω=> ωi) inferInstance
  ...

noncomputable def cycle_posterior
    (env : SixPrimitives.Env S A O) (alg : SixPrimitives.Algorithm A O Sig)
    (K_total L_min k : ℕ) (Theta : Phase2.Trajectory A O (K_total * L_min) →ℝ) :
    Phase2.Trajectory A O (K_total * L_min) →ℝ:=
  let μ:= Phase2.trajMeasure env alg env.hr_meas (K_total * L_min)
  letI : MeasureSpace (Phase2.Trajectory A O (K_total * L_min)) := ⟨μ⟩
  ℙ[Theta | cycle_filtration K_total L_min k]
```

Theorem 8.10 (Accumulation and convergence across cycles)**.** *Under robust X_6, for any integrable quantity Θ measurable with respect to the full horizon:* (i) *if the posterior sequence* $(\text{cycle\_posterior}(k))_k$ *is a martingale with respect to the cycle filtration, it remains one (the property is preserved, not manufac-*

tured); (ii) *if in addition* Θ *is measurable with respect to the filtration's limit, then — given the martingale convergence implication — the posterior converges almost surely to* Θ *itself.*

Lean 4 / Mathlib — Phase5.lean, lines 739--774

```
theorem iet6_accumulative
    (_hRobustX6 : RobustX6_ae env alg L_min)
    (h_int : Integrable Theta (Phase2.trajMeasure env alg env.hr_meas (K_total * L_min)))
    (h_doob_martingale : Integrable Theta (Phase2.trajMeasure env alg env.hr_meas (K_total *
        L_min)) →
      Martingale (fun k => cycle_posterior env alg K_total L_min k Theta)
        (cycle_filtration K_total L_min) (Phase2.trajMeasure env alg env.hr_meas (K_total *
            L_min))) :
    Martingale (fun k => cycle_posterior env alg K_total L_min k Theta)
      (cycle_filtration K_total L_min) (Phase2.trajMeasure env alg env.hr_meas (K_total *
          L_min)) := by
  exact h_doob_martingale h_int

theorem iet6_accumulative_convergence
    (_hRobustX6 : RobustX6_ae env alg L_min)
    (h_int : Integrable Theta (...)) (h_meas : StronglyMeasurable[⨆ k, cycle_filtration
        K_total L_min k] Theta)
    (h_martingale_convergence : Integrable Theta (...) →StronglyMeasurable[...] Theta →
      ∀ᵐ ω∂(...), Tendsto (fun k => cycle_posterior env alg K_total L_min k Theta ω) atTop (
          nhds (Theta ω))) :
    ∀ᵐ ω∂(Phase2.trajMeasure env alg env.hr_meas (K_total * L_min)),
      Tendsto (fun k => cycle_posterior env alg K_total L_min k Theta ω) atTop (nhds (Theta ω
          )) := by
  exact h_martingale_convergence h_int h_meas
```

Implementation Obligation

The filtration and the posterior process are concrete, fully-specified mathematical objects — this is real infrastructure, not a placeholder. What remains deferred is exactly the two facts a real instantiation must supply: that the posterior process actually *is* a martingale under the cycle filtration, and that martingale convergence applies to it (integrability and strong measurability are necessary conditions stated explicitly; supplying them, and the martingale property itself, for a concrete algorithm is the implementer's obligation). Given both, the conclusion — almost-sure convergence of the cycle-by-cycle posterior to the truth — is the precise sense in which evidence compounds across cycles rather than resetting.

The forward and reverse links for $X_6 \to X_1$ themselves follow the same shape as every other link (Phase5.lean, lines 695–705, 776–796): forward relays a deferred `h_pipeline_closes` hypothesis; reverse relays a deferred `h_danger_erodes` hypothesis; and non-reversibility uses the same degenerate-goal-variable construction seen throughout.

Intuition — Investing

A fund that resets its analysis from scratch every quarter never accumulates an edge, no matter how good each quarter's analysis is in isolation. A fund whose quarterly posterior genuinely updates on what came before — a real martingale, not a fresh coin flip dressed up as one — has a process that, by Doob's theorem, must eventually settle down to the truth, provided the underlying quantity is integrable and the process really does have the martingale property. That proviso is not a technicality to wave away: it is the entire content of what "building on accumulated evidence" has to mean for the promise to be more than a slogan.

This completes Part III. The six links, together, give a fully machine-checked logical scaffold for a chain from X_1 through X_6 and back; what is *not* claimed is that any particular algorithm discharges every link's obligations in a general Class $\mathcal{C}$ environment — that construction is environment-specific and is left open, deliberately, for instantiation.

9 Summary of Results

What is proved. Six structural failures (Section 3) each force linear regret on any algorithm lacking the matching primitive, in a minimal environment exhibiting only that one structural failure (Part I, six independent theorems). The six primitives are mutually irreplaceable: for every ordered pair, the algorithm built for one primitive's structural failure is shown to lack a different one when redeployed on that other structural failure's home turf, all thirty pairs following from the same underlying fact — a fixed, non-adaptive policy survives its own structural failure but carries no information about any other (Part II, one master theorem). Six information-theoretic links connect the primitives into a cycle $X_1 \to X_2 \to \cdots \to X_6 \to X_1$; each link is established by a forward result (advance toward full information under stated conditions), a reverse result (an information ceiling under a stated bridging hypothesis), and a non-reversibility result (a fully concrete demonstration that meeting a later primitive's metric without an earlier one is achievable but informationally hollow), and the closing link is grounded in an actual Doob martingale construction (Part III). Every theorem above traces to a specific, named identifier in a machine-checked Lean 4/Mathlib formalization (Appendix A), with every environment's claimed class membership independently verified (Appendix B).

What is not claimed. Several limits are stated explicitly rather than left to be discovered by a careful reader.

- *Necessity is proved on minimal instances, not sufficiency on general ones.* Part I shows that lacking a primitive is fatal in a stripped-down environment exhibiting one structural failure; it does not show that possessing all six primitives is enough to succeed in an arbitrary, more complex member of Class $\mathcal{C}$. The minimality argument (Section 3.3) licenses reading the necessity direction as a transferable floor; it does not license the converse.

- *The literature-comparison claim of Section 4.3 is bounded, not a claim of absolute priority.* It asserts only that no formalization known to this paper combines six independently-proven-necessary structural properties into one machine-checked class with proven mutual independence across all of them — not that six properties are exhaustive, that a harder combination could not be built, or that no such work exists in literature this paper has not encountered.

- *The sequential-dependence chain is a verified scaffold, not a closed theorem about any concrete algorithm.* IET 1, 2, 3, and 6 name a concrete possession predicate; IET 4 and IET 5 leave the possession predicate itself abstract. Every forward and reverse link carries at least one hypothesis — named explicitly in an Implementation Obligation box at the point it is used — that this paper does not discharge for any specific environment. No claim is made that robustly possessing X_i automatically produces the conditions a concrete instantiation of link i would need.

- *Two genuinely different notions share the name "conditional entropy."* Phase 1's `condEntropyOf` and Phase2CMI's `condEntropy` are independently defined and used in different parts of the paper (Remark 5.5); neither is derived from the other.

- *X_1's necessity proof uses a total-variation surrogate, not Shannon mutual information*, despite the naming convention inherited from Phase 0 (Remark 6.4).

- *X_2's necessity hypothesis is a deliberate strengthening of F_2, not a literal instance of it* (Remark 6.10), because the unconditional version is vacuous in an absorbing-trap environment.

- *The fourteen unused Phase 1 tools are positioned, not proven to work.* Remark 5.6 identifies which Part III obligation each is plausibly suited to discharging; none of them has actually been used to discharge it. Confirming that fit is exactly the constructive work this paper leaves open, not a claim made here.

10 Conclusion

The six primitives were chosen to be stated abstractly enough to survive outside the reinforcement-learning setting that motivated them — a hidden objective worth identifying, a mistake worth never repeating, a plateau worth pushing past, a decision worth eventually making, a constraint worth never trading away, and evidence worth retiring once it goes stale recur in fund management, operations, consulting, and public policy under different names but the same shape. The minimality principle (Section 3.3) is what makes that portability more than a rhetorical flourish: each necessity result is proved against the least elaborate environment that still forces the failure, so the floor it establishes persists into every harder environment containing that instance as a special case.

What this paper deliberately leaves open is the constructive question: exhibiting a concrete algorithm, in a concrete richer member of Class $\mathcal{C}$, that discharges every Implementation Obligation in Part III simultaneously. That construction is environment-specific by nature, and collapsing it into a single abstract theorem would either trivialize it or smuggle in unstated assumptions about the environment — exactly the failure mode this paper's machine-checked discipline was adopted to avoid. The verified scaffold is offered as the foundation that construction would stand on, not as a substitute for it.

References

George A. Akerlof. The market for 'lemons': Quality uncertainty and the market mechanism. *The Quarterly Journal of Economics*, 84(3):488–500, 1970.

Kenneth J. Arrow and Gerard Debreu. Existence of an equilibrium for a competitive economy. *Econometrica*, 22(3):265–290, 1954.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. *Journal of the ACM*, 65(3):1–55, 2018.

Austin Coursey, Abel Diaz-Gonzalez, Marcos Quinones-Grueiro, and Gautam Biswas. Safe continual reinforcement learning in non-stationary environments. *arXiv preprint arXiv:2604.19737*, 2026.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer International Publishing, 2021.

Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT 2011)*, pages 174–188. Springer, 2011.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.

Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped DQN. In *Advances in Neural Information Processing Systems 29*, pages 4026–4034, 2016.

Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, Tor Lattimore, Csaba Szepesvári, Satinder Singh, Benjamin Van Roy, Richard Sutton, David Silver, and Hado van Hasselt. Behaviour suite for reinforcement learning. In *International Conference on Learning Representations*, 2020.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381. ACM, 2020.

Benjamin Plaut, Hanlin Zhu, and Stuart Russell. Avoiding catastrophe in online learning by asking for help. In *Proceedings of the International Conference on Machine Learning (ICML 2025)*, 2025.

Daniel Russo and Benjamin Van Roy. An information-theoretic analysis of Thompson sampling. *Journal of Machine Learning Research*, 17(1):2442–2471, 2016.

Alexander L. Strehl and Michael L. Littman. An analysis of model-based interval estimation for Markov decision processes. *Journal of Computer and System Sciences*, 74(8):1309–1331, 2008.

Timofey Tomashevskiy. Safe continual reinforcement learning methods for nonstationary environments: Towards a survey of the state of the art. *arXiv preprint arXiv:2601.05152*, 2026.

John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.

Appendices

A Lean ↔ Paper Correspondence Table

Every numbered Definition, Theorem, Lemma, and Example in this paper is listed below against its exact Lean identifier and location. This is the table that makes the "machine-verified" claim auditable: anyone may open the cited file at the cited line and check that the paper's statement is not a paraphrase but the literal formalization. The complete formalization is available at https://github.com/M-Ismail-ZA/IsmailsPrimitives.

Setup and Environment Class (Sections 2–3)

Paper item	Lean identifier	File	Lines
Definition 2.1	`Env`	Phase0.lean	32–45
Definition 2.2	`EnvIsMarkov`, `EnvIsDeterministic`	Phase2.lean	29–58
Definition 2.3	`Algorithm`	Phase0.lean	50–59
Definition 2.4	`Trajectory`	Phase2.lean	202
Definition 2.5	`trajMeasure`	Phase2.lean	204–211
Definition 2.7	`algValue`, `optValue`	Phase2.lean	1600–1614
Definition 2.8	`regret`, `SublinearRegret`, `LinearRegret`	Phase2.lean	1615–1632
Definition 2.10	$\texttt{LacksX}_1$–$\texttt{LacksX}_6$	Phase0.lean	69–102
Definition 2.11	$\texttt{PossessesX}_i$, $\texttt{RobustX}_i$ (i=1–6)	Phase0.lean	110–156
Definition 3.1	$\texttt{HasP}_1$–$\texttt{HasP}_6$	Phase0.lean	167–213
Definition 3.2	`InClassC`	Phase0.lean	215–225

Standard Tools (Section 5)

Paper item	Lean identifier	File	Lines
Definition 5.1	`condEntropy`, `entropy`	Phase2CMI.lean	24–34
Lemma 5.2	`condEntropy_le_entropy`, `condEntropy_of_deterministic`	Phase2CMI.lean	158–173
Definition 5.3	`binEntropy`, `binEntropyInv`, `binEntropyInv_spec`	Phase1.lean	215–221, 436–451
Lemma 5.4	`fano_binary_error_lower_bound`	Phase2CMI.lean	974–982
Remark 5.5	`condEntropyOf`	Phase1.lean	51–57

Part I — Ismail's Proof of Primitive Necessity (Section 6)

Paper item	Lean identifier	File	Lines
Definition 6.1	`BanditParam`, $\texttt{env}_1\texttt{\_0}$, $\texttt{env}_1\texttt{\_1}$	Phase2.lean	1702–1730
Definition 6.3	`miSeq`	Phase3.lean	88–97
Theorem 6.5	`necessity_x1`	Phase3.lean	122–561
Definition 6.7	$\texttt{env}_2$, $\texttt{env}_2\texttt{\_trap}$	Phase2.lean	1934–1953
Definition 6.9	`survivedToStep`, `survivalProbSeq`, `dangerAndSurvivalProbSeq`, `condDangerProbSeq`	Phase2.lean	4806–4838
Theorem 6.11	`necessity_x2`	Phase3.lean	931–1222
Definition 6.13	$\texttt{env}_3\texttt{\_trans}$, $\texttt{env}_3$	Phase2.lean	2095–2112
Theorem 6.15	`necessity_x3`	Phase3.lean	1556–1633
Definition 6.18	$\texttt{env}_4$	Phase2.lean	3742–3748
Definition 6.20	`condEntSeqX4`	Phase3.lean	1653–1660
Theorem 6.21	`necessity_x4_explicit_bound`	Phase3.lean	1662–1829
Theorem 6.22	`necessity_x4`	Phase3.lean	1831–1857
Definition 6.24	$\texttt{feasSet}_5$, $\texttt{penalty}_5$, $\texttt{env}_5$	Phase2.lean	3838–3847
Theorem 6.26	`necessity_x5_explicit_bound`, `necessity_x5`	Phase3.lean	1885–2069
Definition 6.28	$\texttt{env}_6\texttt{\_nonstat}$	Phase3.lean	2078–2094
Definition 6.30	`staleProbSeq`	Phase2.lean	1512–1518
Theorem 6.31	`necessity_x6_explicit_bound`, `necessity_x6`	Phase3.lean	2437–2598

Part II — Ismail's Proof of Primitive Independence (Section 7)

Paper item	Lean identifier	File	Lines
Template/policies	canonical_challenger, alg1_bayes–alg6_shortMem, pol0, pol1, polR, pol_eps	Phase4.lean	28–86
Example 7.2	alg2_lacks_x1	Phase4.lean	111–162
Example 7.3	canonical_challenger_lacks_x2, alg1_lacks_x2	Phase4.lean	746–769
Table 3, row $\mathcal{E}_1$	alg2_lacks_x1–alg6_lacks_x1	Phase4.lean	103–407
Table 3, row $\mathcal{E}_2$	alg1_lacks_x2, alg3_lacks_x2–alg6_lacks_x2	Phase4.lean	663–769
Table 3, row $\mathcal{E}_3$	alg1_lacks_x3, alg2_lacks_x3, alg4_lacks_x3–alg6_lacks_x3	Phase4.lean	773–931
Table 3, row $\mathcal{E}_4$	canonical_challenger_lacks_x4, alg1_lacks_x4–alg3_lacks_x4, alg5_lacks_x4, alg6_lacks_x4	Phase4.lean	1083–1137
Table 3, row $\mathcal{E}_5$	alg1_lacks_x5–alg4_lacks_x5, alg6_lacks_x5	Phase4.lean	1149–1399
Table 3, row $\mathcal{E}_6$	deterministic_zero_lacks_x6, alg1_lacks_x6–alg5_lacks_x6	Phase4.lean	1403–1475
Theorem 7.4	mutual_independence	Phase4.lean	1480–1532

Part III — Ismail's Formally-Verified Bridge Towards Sufficiency (Section 8)

Paper item	Lean identifier	File	Lines
Infrastructure	MI_Summary_Seq, IsSufficientSummary	Phase5.lean	27–62
Theorem 8.1	iet1_forward	Phase5.lean	222–229
Theorem 8.2	iet1_reverse	Phase5.lean	231–237
Theorem 8.3	iet1_non_reversible	Phase5.lean	239–264
Definition 8.4	RobustX2, RobustX2_general	Phase5.lean	101–124
Definition 8.5	bridge_count_rv, RobustX3_ae	Phase5.lean	126–150
Theorem 8.6	iet2_forward	Phase5.lean	278–307
Theorem 8.7	iet2_reverse	Phase5.lean	310–361
Theorem 8.8	iet2_non_reversible	Phase5.lean	362–391
Theorem 8.9	iet3_forward	Phase5.lean	404–416
IET 3 reverse, non-reversible	iet3_reverse, iet3_non_reversible	Phase5.lean	418–521
IET 4 (all three)	iet4_forward, iet4_reverse, iet4_non_reversible	Phase5.lean	553–612
IET 5 (all three)	iet5_forward, iet5_reverse, iet5_non_reversible	Phase5.lean	624–682
Infrastructure	cycle_filtration, cycle_posterior	Phase5.lean	707–737
Theorem 8.10	iet6_accumulative, iet6_accumulative_convergence	Phase5.lean	739–774
IET 6 forward, reverse, non-rev.	iet6_forward, iet6_reverse, iet6_non_reversible	Phase5.lean	695–705, 776–796

B Verification that $\mathcal{E}_i \in \mathcal{C}$

Each environment used in Part I is verified, directly and only once, to actually exhibit the structural property (Definition 3.1) it is claimed to witness. These six short proofs are the complete discharge of every "class membership" remark made in Section 6.

Lean 4 / Mathlib — Phase2.lean, lines 1731--1738

```
theorem env₁_0_hasP₁ (bp : BanditParam) : SixPrimitives.HasP₁ (env₁_0 bp) (env₁_1 bp) := by
  refine ⟨(), 0, 1, by decide, ?_, ?_⟩
  ·intro a
    fin_cases a <;> simp [env₁_0]
    linarith [bp.hΔ0]
  ·intro a
    fin_cases a <;> simp [env₁_1]
    linarith [bp.hΔ0]
```

Lean 4 / Mathlib — Phase2.lean, lines 1956--1974

```
theorem env₂_hasP₂ (K : ℕ) (hK : 0 < K) : SixPrimitives.HasP₂ (env₂ K hK) (env₂_trap K) := by
  refine ⟨⟨⟨K + 1, by omega⟩, rfl⟩, ?_, ?_, ?_, ?_⟩
  ·intro h
    have : (⟨0, by omega⟩ : Fin (K + 2)) ∈Set.univ := Set.mem_univ _
    rw [← h] at this; simp [env₂_trap] at this
  ·intro s hs a
    have hval : s = ⟨K + 1, by omega⟩ := hs
    subst hval
    simp only [env₂, Kernel.deterministic_apply,
               show ¬(K + 1 < K) from Nat.not_lt.mpr (Nat.le_refl K |>.trans (Nat.le_succ K))]
    simp [env₂_trap]
  ·intro s hs a
    have hval : s = ⟨K + 1, by omega⟩ := hs; subst hval
    simp [env₂, show ¬(K + 1 < K) from by omega]
  ·refine ⟨⟨0, by omega⟩, by simp [env₂_trap], 1, ?_⟩
    simp only [env₂, Kernel.deterministic_apply,
               show (⟨0, by omega⟩ : Fin (K + 2)).val < K from hK]
    simp only [show (1 : Fin 2) ≠0 from by decide]
    simp [env₂_trap]
```

Lean 4 / Mathlib — Phase2.lean, lines 2114--2160

```
theorem env₃_hasP₃ (p : ℝ) (hp0 : 0 < p) (hp1 : p ≤1) :
    SixPrimitives.HasP₃ (env₃ p hp0.le hp1) ({0} : Set (Fin 2)) ({1} : Set (Fin 2)) := by
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_⟩
  ·exact Set.disjoint_singleton.mpr (by decide)
  ·exact Set.singleton_nonempty 0
  ·exact Set.singleton_nonempty 1
  ·intro s hs a h_greedy
    simp only [Set.mem_singleton_iff] at hs
    subst hs
    have h_a_eq_0 : a = 0 := by
      revert h_greedy; fin_cases a
      ·intro _; rfl
      ·intro h_greedy
```

```
      have h_greedy_0 := h_greedy 0
      simp only [env₃] at h_greedy_0
      norm_num at h_greedy_0
  subst h_a_eq_0
  simp only [env₃, env₃_trans, Kernel.coe_mk]
  have h01 : (0 : Fin 2) ≠1 := by decide
  simp only [h01, ite_false, ite_true]
  exact MeasureTheory.Measure.dirac_apply_of_mem (Set.mem_singleton 0)
·refine ⟨0, Set.mem_singleton 0, 1, ?_⟩
  simp only [env₃, env₃_trans, Kernel.coe_mk]
  have h1 : (0 : Fin 2) = 1 ↔False := by decide
  have h2 : (1 : Fin 2) = 0 ↔False := by decide
  simp only [h1, h2, ite_false]
  have h_meas : Measurable (fun b : Bool => if b then (1 : Fin 2) else 0) :=
    measurable_of_finite _
  rw [MeasureTheory.Measure.map_apply h_meas (measurableSet_singleton 1)]
  have h_pre : (fun b : Bool => if b then (1 : Fin 2) else 0) ⁻¹' {1} = {true} := by
    ext b; cases b <;> simp
  rw [h_pre]
  rw [(PMF.bernoulli ⟨p, hp0.le⟩ hp1).toMeasure_apply (measurableSet_singleton true)]
  simp [tsum_fintype, Set.indicator_apply, PMF.bernoulli_apply]
  exact hp0
·refine ⟨1/2, 1, ?_, ?_, by norm_num⟩
  ·intro s hs a
    simp only [Set.mem_singleton_iff] at hs; subst hs
    have h01 : (0 : Fin 2) ≠1 := by decide
    simp only [env₃, h01, ite_false]
    split_ifs
    ·norm_num
    ·linarith [hp1]
  ·intro s hs; use 0
    simp only [Set.mem_singleton_iff] at hs; subst hs
    simp only [env₃, ite_true, le_refl]
```

Lean 4 / Mathlib — Phase2.lean, lines 3760--3766

```
theorem env₄_hasP₄ : SixPrimitives.HasP₄ env₄ () (0 : Fin 2) SixPrimitives.
    VisitFrequencyAtLeast := by
  refine ⟨1, 1, one_pos, one_pos, ?_, ?_⟩
  ·intro a ha
    fin_cases a
    ·exact absurd rfl ha
    ·simp [env₄]
  ·exact SixPrimitives.visitFrequencyAtLeast_of_unique_state env₄ env₄_isDet 1 le_rfl
```

Lean 4 / Mathlib — Phase2.lean, lines 3862--3870

```
theorem env₅_hasP₅ : SixPrimitives.HasP₅ env₅ feasSet₅ penalty₅ := by
  refine ⟨by norm_num [penalty₅], ?_, ?_⟩
  ·constructor
    ·exact Set.subset_univ _
    ·intro h
      have h12 : (1/2 : ℝ) ∈feasSet₅ := h (Set.mem_univ _)
      norm_num [feasSet₅, Set.mem_union, Set.mem_Icc] at h12
```

```
  ·intro _ a ha
    simp only [env₅, if_neg ha, le_refl]
```

Lean 4 / Mathlib — Phase2.lean, lines 3965--3987

```
noncomputable def env₆_family (T : ℕ) (Δ : ℝ) (hΔ0 : 0 < Δ) (hΔ4 : Δ≤1/4)
    (n : ℕ) : SixPrimitives.Env Unit (Fin 2) Bool :=
  let postCP : Prop := T / 2 < n
  { trans := Kernel.const _ (Measure.dirac ())
    obs := { toFun := fun (_, a) =>
                 if (a = (0 : Fin 2)) = postCP
                 then bernoulliMeasure (1/2 + Δ) (by linarith) (by linarith)
                 else bernoulliMeasure (1/2 - Δ) (by linarith [hΔ4]) (by linarith [hΔ0])
               measurable' := measurable_of_finite _ }
    r := fun (_, a) =>
              if (a = (0 : Fin 2)) = postCP then 1/2 + Δelse 1/2 - Δ
    hr_meas := measurable_of_finite _
    μ₀:= Measure.dirac ()
    hμ₀ := inferInstance }

theorem env₆_hasP₆ (T : ℕ) (_hT : 0 < T) (Δ : ℝ) (hΔ0 : 0 < Δ) (hΔ4 : Δ≤1/4) :
    SixPrimitives.HasP₆ (env₆_family T ΔhΔ0 hΔ4) (T / 2) := by
  refine ⟨(), 0, Or.inl ?_⟩
  simp only [env₆_family]
  simp only [show ¬(T / 2 < T / 2) from Nat.lt_irrefl _,
             show T / 2 < T / 2 + 1 from Nat.lt_succ_self _]
  norm_num
  linarith
```

Remark B.1 (Two environments, one structural failure — resolved)**.** *This is the literal sequence-of-environments construction referenced in Remark 6.29:* ***env₆_family*** *differs at $n = \tau$ exactly as* HasP_6 *requires, while the necessity proof itself runs against* ***env₆_nonstat*** *(Definition 6.28), which packages the identical structural failure with time folded into the state. Both are shown here in full so the equivalence is checkable directly rather than asserted.*